ES-CS301

Data Structure & Algorithms

Combined Module Notes

Modules 1 – 4

Semester III • ECE • 3 Credits

CGEC | B.Tech ECE (2023–27)

Disclaimer / Study Guide Notice:
These are condensed **Short Notes** focusing on core theory, key formulas, block/circuit diagrams, and standard viva Q&As for university exam revision. Exhaustive numerical practice problems and derivation edge cases are not covered. This serves as a quick-revision reference guide for students.

Biraj Sarkar

IN ASSOCIATION WITH

MAKAUT Wingman • **MAKAUT Future Minds**

Cooch Behar Government Engineering College

Exam-Ready Notes

Contents

Module 1: Elementary Data Organizations and Terminologies 5

1 Elementary Data Organizations and Terminologies . . . 5

1.1 Basic Data Organization Terminologies . . . 5

1.2 Taxonomy of Data Structures . . . 5

1.3 Fundamental Data Structure Operations . . . 6

1.4 Array Memory Representations and Address Calculation . . . 6

2 Algorithm Analysis and Performance Metrics . . . 8

2.1 Performance Metrics: Time and Space Complexity . . . 8

2.2 Frequency Count Method . . . 8

3 Asymptotic Notations and Growth of Functions . . . 9

3.1 Little-o and Little-omega Notations . . . 9

3.2 Hierarchy and Properties of Growth Rates . . . 10

3.3 The Time-Space Trade-Off . . . 10

4 Searching Techniques and Algorithmic Complexity . . . 10

4.1 Linear (Sequential) Search . . . 10

4.2 Binary Search Technique . . . 11

4.3 Comparison: Linear Search vs Binary Search . . . 12

Module 2: Abstract Data Type Stack and Fundamental Operations 16

1 Abstract Data Type Stack and Fundamental Operations . . . 16

1.1 Array Implementation of Stack . . . 16

2 Applications of Stacks . . . 17

2.1 Arithmetic Expression Formats and Operator Precedence . . . 17

2.2 Infix to Postfix Conversion Algorithm . . . 18

2.3 Evaluation of Postfix Expressions . . . 19

2.4 Infix to Prefix Conversion and Prefix Evaluation . . . 20

2.5 Balanced Delimiter Checking and Call Stack Simulation . . . 21

3 Abstract Data Type Queue and Linear Queue . . . 21

3.1 Array Implementation of Linear Queue . . . 21

4 Advanced Queue Architectures . . . 22

4.1 Circular Queue . . . 22

4.2 Priority Queue . . . 23

4.3 Double-Ended Queue (DEQUE) . . . 24

Module 3: Linked Lists Fundamentals and Singly Linked List (SLL) 28

1 Linked Lists Fundamentals and Singly Linked List (SLL) . . . 28

1.1 Limitations of Arrays vs Advantages of Linked Lists . . . 28

1.2 Singly Linked List Algorithmic Operations . . . 29

2 Linked Representation of Stack and Queue & Header Nodes . . . 29
2.1 Linked Stack and Linked Queue . . . 29
2.2 Header Nodes in Linked Lists . . . 30
3 Doubly Linked Lists (DLL) and Circular Linked Lists (CLL) . . . 30
3.1 Doubly Linked List (DLL) . . . 30
3.2 Circular Linked Lists (CLL) . . . 30
3.3 Polynomial Representation and Addition using Linked Lists . . . 31
4 Tree Terminologies and Binary Tree Foundations . . . 31
4.1 Mathematical Properties of Binary Trees . . . 31
4.2 Binary Tree Classifications . . . 32
4.3 Binary Tree Traversals . . . 32
5 Threaded Binary Tree . . . 33
6 Binary Search Tree (BST) . . . 33
6.1 BST Operations and Recursive Algorithms . . . 33
7 AVL Tree (Self-Balancing Binary Search Tree) . . . 34
7.1 AVL Rebalancing Rotations . . . 34
8 Applications of Binary Trees: Expression Trees and Huffman Coding . . . 35
8.1 Expression Trees . . . 35
8.2 Huffman Coding (Greedy Data Compression) . . . 35
9 Multi-Way Search Trees: B-Tree and B+ Tree . . . 36
9.1 B-Tree of Order m . . . 36
9.2 B+ Tree and Comparison with B-Tree . . . 37

Module 4: Classification and Criteria of Sorting Algorithms 41

1 Classification and Criteria of Sorting Algorithms . . . 41
2 Elementary Sorting Algorithms . . . 42
2.1 Selection Sort . . . 42
2.2 Bubble Sort . . . 42
2.3 Insertion Sort . . . 42
3 Advanced Divide-and-Conquer & Tree-Based Sorting . . . 43
3.1 Quick Sort (Partition-Exchange Sort) . . . 43
3.2 Merge Sort . . . 43
3.3 Heap Sort . . . 44
3.4 Comprehensive Sorting Comparison Matrix . . . 45
4 Hashing Techniques and Collision Resolution . . . 45
4.1 Standard Hash Functions . . . 45
4.2 Collision Resolution Techniques . . . 46
5 Graph Theory, Representations and Traversals . . . 46
5.1 Graph Storage Representations . . . 47

© Biraj

5.2 Graph Search and Traversal Algorithms 47

© Biraj

Module 1

Elementary Data Organizations and Terminologies

1. Elementary Data Organizations and Terminologies

Definition: Data Structure and Abstract Data Type (ADT)

A **data structure** is a specialized mathematical or logical organization of data items alongside a set of well-defined operations, designed to facilitate efficient storage, retrieval, and algorithmic modification in computer memory.

An **Abstract Data Type (ADT)** is an axiomatic mathematical specification of a data structure that defines:
1. The domain of data elements stored, and
2. The legal operations performed upon them together with their formal behavioral contracts,

independent of any underlying hardware architecture, compiler implementation, or memory layout.

1.1 Basic Data Organization Terminologies

Data items are structured hierarchically in memory from physical bits to logical models:

- **Data Item:** A single unit of value (e.g., an integer or a character).
- **Elementary vs Group Data Item:** An elementary item cannot be decomposed further (e.g., student ID). A group item can be divided into sub-items (e.g., full name divided into first, middle, and last names).
- **Entity and Attribute:** An entity represents a concrete real-world object (e.g., an employee). An attribute is an observable property or characteristic of that entity (e.g., age, salary).
- **Record:** A composite collection of heterogeneous data attributes representing a single entity.
- **File:** A collection of structurally identical records representing an entity set.

1.2 Taxonomy of Data Structures

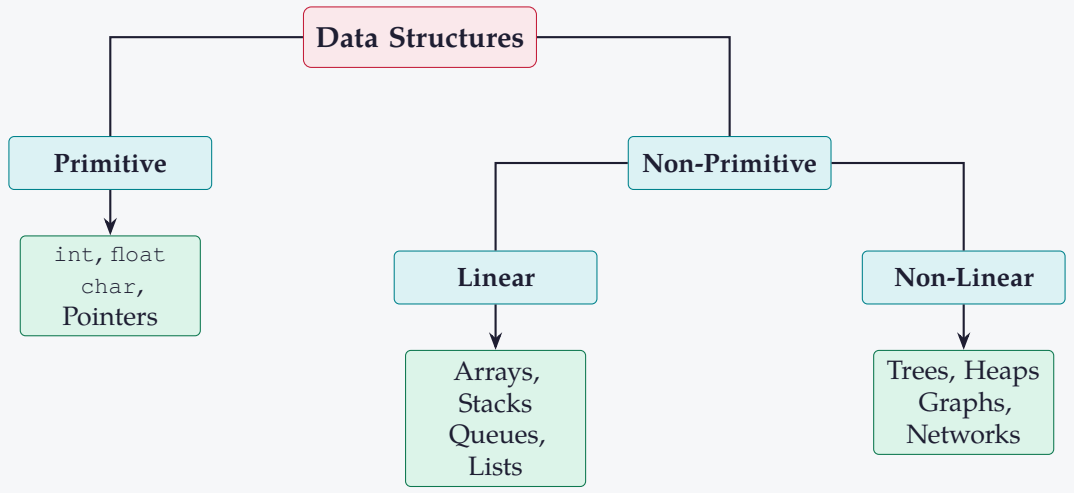
Data structures are classified across multiple functional dimensions:

1. **Primitive vs Non-Primitive:** Primitive types are directly manipulated by underlying machine instructions (e.g., `int`, `float`, `char`, pointers). Non-primitive types are derived composite structures (e.g., arrays, lists, trees).

© Biraj

2. **Linear vs Non-Linear:** In a linear data structure, elements form a strictly sequential 1-to-1 ordering where every element (except first and last) has a unique predecessor and successor (e.g., Array, Linked List, Stack, Queue). In non-linear data structures, elements exhibit hierarchical (1-to-many) or interconnected (many-to-many) relationships (e.g., Trees, Graphs).
3. **Static vs Dynamic:** Static structures maintain a fixed memory allocation determined at compile time (e.g., static arrays). Dynamic structures allocate and deallocate physical memory blocks at runtime via system heaps (e.g., Linked Lists, dynamic BSTs).
4. **Homogeneous vs Heterogeneous:** Homogeneous structures enforce identical data types for all elements (e.g., arrays). Heterogeneous structures permit dissimilar types (e.g., C `struct`, records).
5. **Persistent vs Ephemeral:** Ephemeral structures destroy earlier versions upon modification; persistent structures preserve previous historical states after updates.

Structural Hierarchy and Taxonomy of Data Structures



1.3 Fundamental Data Structure Operations

Every computational structure supports standard operational primitives:

1. **Traversing:** Accessing each stored element exactly once to process or display data.
2. **Insertion:** Adding a new data element at a designated index, boundary, or ordered position.
3. **Deletion:** Removing an existing data item and releasing or compacting memory.
4. **Searching:** Finding the memory address or index of an element satisfying a target search key.
5. **Sorting:** Permuting elements into a monotonic ascending or descending collation order.
6. **Merging:** Combining two distinct ordered datasets into a unified ordered sequence.

1.4 Array Memory Representations and Address Calculation

An array is a homogeneous, contiguous collection of items accessed via integer offsets.

Let:

© Biraj

- $B =$ Base Address (memory address of the first byte of the array),
- $W =$ Size of each element in bytes (word size),
- $LB =$ Lower Bound of indices (default is 0 in C, 1 in mathematical texts).

Address Calculation Formulas for Multi-Dimensional Arrays

1. One-Dimensional Array $A[LB \ldots UB]$:

$$\text{Loc}(A[i]) = B + W \cdot (i - LB) \tag{1}$$

2. Two-Dimensional Array $A[LB_1 \ldots UB_1,\ LB_2 \ldots UB_2]$:
Let $N_1 = UB_1 - LB_1 + 1$ (total rows) and $N_2 = UB_2 - LB_2 + 1$ (total columns).

- **Row-Major Order (RMO)** (consecutive row elements stored adjacently):

$$\text{Loc}(A[i][j]) = B + W \cdot \Big[(i - LB_1) \cdot N_2 + (j - LB_2)\Big] \tag{2}$$

- **Column-Major Order (CMO)** (consecutive column elements stored adjacently):

$$\text{Loc}(A[i][j]) = B + W \cdot \Big[(j - LB_2) \cdot N_1 + (i - LB_1)\Big] \tag{3}$$

Exam Numerical: 2D Array Address Calculation

Problem: An array $A[-5 \ldots 15,\ 10 \ldots 40]$ is allocated in byte-addressed RAM starting at base address $B = 1024$. Each element occupies $W = 4$ bytes. Determine the memory location of element $A[3][25]$ under both Row-Major and Column-Major storage schemes.

Solution:

- $LB_1 = -5,\ UB_1 = 15 \implies N_1 = 15 - (-5) + 1 = 21$ rows.
- $LB_2 = 10,\ UB_2 = 40 \implies N_2 = 40 - 10 + 1 = 31$ columns.
- Row offset: $i - LB_1 = 3 - (-5) = 8$.
- Column offset: $j - LB_2 = 25 - 10 = 15$.

1. Under Row-Major Order (RMO):

$$\begin{aligned}\text{Loc}(A[3][25]) &= 1024 + 4 \cdot [8 \cdot 31 + 15] = 1024 + 4 \cdot [248 + 15] \\ &= 1024 + 4 \cdot 263 = 1024 + 1052 = \mathbf{2076}\end{aligned}$$

2. Under Column-Major Order (CMO):

$$\begin{aligned}\text{Loc}(A[3][25]) &= 1024 + 4 \cdot [15 \cdot 21 + 8] = 1024 + 4 \cdot [315 + 8] \\ &= 1024 + 4 \cdot 323 = 1024 + 1292 = \mathbf{2316}\end{aligned}$$

2. Algorithm Analysis and Performance Metrics

© Biraj

Definition: Algorithm Criteria and Formal Characteristics

An **algorithm** is a finite, unambiguous, step-by-step computational procedure that accepts an initial input set and transforms it into a deterministic output set. Formally, every valid algorithm must satisfy:

1. **Input:** Generates output from zero or more well-defined external inputs.
2. **Output:** Produces at least one quantifiable mathematical or logical result.
3. **Definiteness:** Every operational step is completely clear, rigorous, and unambiguous.
4. **Finiteness:** Terminates strictly after a finite number of executable instructions for all valid inputs.
5. **Effectiveness:** Every instruction is basic enough to be executed exactly in finite time (feasible on physical hardware).

2.1 Performance Metrics: Time and Space Complexity

Algorithmic efficiency is measured independently of machine clock speed, compiler optimizations, or operating system jitter:

- **Space Complexity $S(P)$:** Total memory consumed by program P over its execution lifecycle:

$$S(P) = c + S_{\text{variable}}(I) \tag{4}$$

  where c is the fixed part (instruction space, constants, scalar variables) and $S_{\text{variable}}(I)$ is the instance-dependent variable part (dynamically allocated memory, recursion stack activation frames).

- **Time Complexity $T(P)$:** Total computational operations or instruction steps executed as a mathematical function of input problem size n.

2.2 Frequency Count Method

The frequency count determines how many times each statement executes:

Step-Count Analysis of Nested Loops

Consider the matrix multiplication algorithm for two $n \times n$ matrices:

```
for (i = 0; i < n; i++)                    // Executes n + 1 times
    for (j = 0; j < n; j++)                // Executes n * (n + 1) times
    {
        C[i][j] = 0;                       // Executes n * n times
        for (k = 0; k < n; k++)            // Executes n * n * (n + 1) times
            C[i][j] += A[i][k]*B[k][j]; // Executes n * n * n times
    }
```

Total inner statement execution count: $f(n) = n^3$. As $n \to \infty$, the cubic term dominates all lower-order terms, yielding a time complexity of $\Theta(n^3)$.

© Biraj

3. Asymptotic Notations and Growth of Functions

Formal Definitions of Asymptotic Bounds

Let $f(n)$ and $g(n)$ be asymptotically non-negative functions defined on the set of natural numbers $\mathbb{N}$.

1. **Big-O Notation ($\mathcal{O}$ — Asymptotic Upper Bound):**

$$\mathcal{O}(g(n)) = \{f(n) : \exists\, c > 0,\ n_0 \in \mathbb{N} \text{ such that } 0 \leq f(n) \leq c \cdot g(n)\ \ \forall\, n \geq n_0\} \tag{5}$$

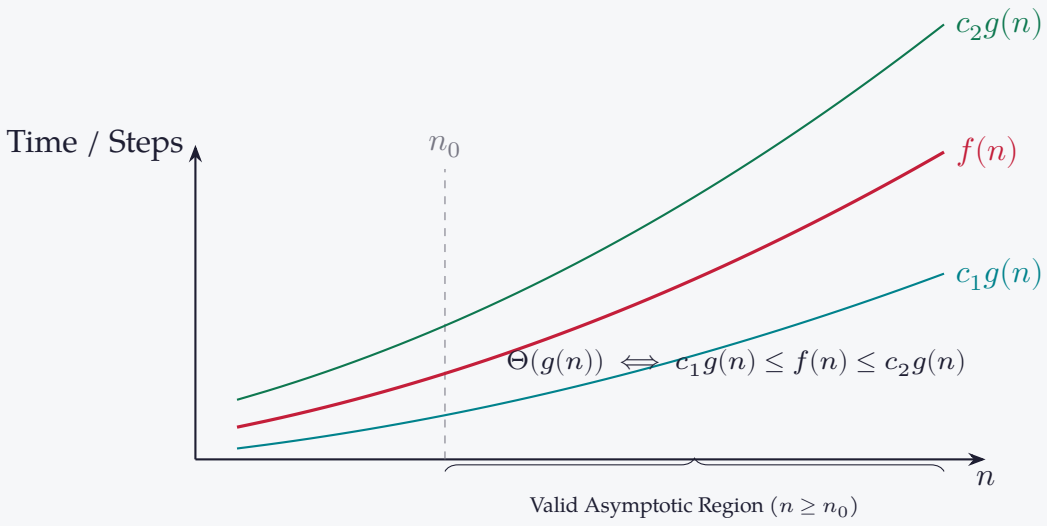
2. **Big-Omega Notation (Ω — Asymptotic Lower Bound):**

$$\Omega(g(n)) = \{f(n) : \exists\, c > 0,\ n_0 \in \mathbb{N} \text{ such that } 0 \leq c \cdot g(n) \leq f(n)\ \ \forall\, n \geq n_0\} \tag{6}$$

3. **Big-Theta Notation (Θ — Asymptotically Tight Bound):**

$$\Theta(g(n)) = \{f(n) : \exists\, c_1, c_2 > 0,\ n_0 \in \mathbb{N} \text{ s.t. } 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n)\ \ \forall\, n \geq n_0\} \tag{7}$$

Geometric Interpretation of Asymptotic Notations



3.1 Little-o and Little-omega Notations

- **Little-o Notation (o — Strict Upper Bound):**

$$f(n) = o(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0 \tag{8}$$

  Unlike $\mathcal{O}$, $f(n)$ cannot be asymptotically equal to $g(n)$ (e.g., $2n = o(n^2)$, but $2n \neq o(n)$).

- **Little-omega Notation (ω — Strict Lower Bound):**

$$f(n) = \omega(g(n)) \iff \lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty \tag{9}$$

© Biraj

3.2 Hierarchy and Properties of Growth Rates

Standard asymptotic complexities scale monotonically:

$$\begin{aligned}\mathcal{O}(1) < \mathcal{O}(\log\log n) < \mathcal{O}(\log n) < \mathcal{O}(\sqrt{n}) < \mathcal{O}(n) \\ < \mathcal{O}(n\log n) < \mathcal{O}(n^2) < \mathcal{O}(n^3) < \mathcal{O}(2^n) < \mathcal{O}(n!) < \mathcal{O}(n^n)\end{aligned} \tag{10}$$

Fundamental Algebraic Properties:

- **Transitivity:** If $f(n) = \Theta(g(n))$ and $g(n) = \Theta(h(n))$, then $f(n) = \Theta(h(n))$.
- **Symmetry:** $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$.
- **Transpose Symmetry:** $f(n) = \mathcal{O}(g(n)) \iff g(n) = \Omega(f(n))$.
- **Theorem:** $f(n) = \Theta(g(n)) \iff f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$.

3.3 The Time-Space Trade-Off

In computational engineering, the **Time-Space Trade-off** dictates that algorithmic runtime can often be reduced at the expense of higher auxiliary memory usage, and vice-versa.

- **Memory-Heavy, Fast Execution:** Hash tables provide $\mathcal{O}(1)$ average lookup by utilizing extra array buckets. Pre-computed lookup tables or memoization arrays in Dynamic Programming trade space to eliminate redundant exponential recursive calls.
- **Memory-Light, Slow Execution:** In-place algorithms with $\mathcal{O}(1)$ auxiliary space (e.g., linear scanning or recomputing values on demand) save RAM at the cost of $\mathcal{O}(n)$ or $\mathcal{O}(n^2)$ processing delays.

4. Searching Techniques and Algorithmic Complexity

4.1 Linear (Sequential) Search

Linear search scans an unsorted array sequentially from index 0 to $n-1$, checking each element against target key K.

Linear Search Algorithm and Complexity Proof

Algorithm LinearSearch(A, n, K)**:**
1. For $i = 0$ to $n-1$ do:
2. If $A[i] == K$, return index i.
3. Return -1 (Search Key Not Found).

Complexity Derivation:

- **Best-Case Time Complexity:** Target key is at the first index ($A[0] == K$). Requires 1 comparison:

$$T_{\text{best}}(n) = \mathcal{O}(1) \tag{11}$$

- **Worst-Case Time Complexity:** Target key is at the last index ($A[n-1]$) or absent from array. Requires n comparisons:

$$T_{\text{worst}}(n) = \mathcal{O}(n) \tag{12}$$

© Biraj

- **Average-Case Time Complexity:** Assuming uniform distribution where key is present with probability p, and equally likely at any position $(1/n)$:

$$T_{\text{avg}}(n) = \sum_{i=1}^{n} i \cdot P(\text{found at } i) = \sum_{i=1}^{n} i \cdot \frac{1}{n} = \frac{1}{n} \cdot \frac{n(n+1)}{2} = \frac{n+1}{2} \implies \mathcal{O}(n) \quad (13)$$

- **Auxiliary Space:** $\mathcal{O}(1)$ (in-place search).

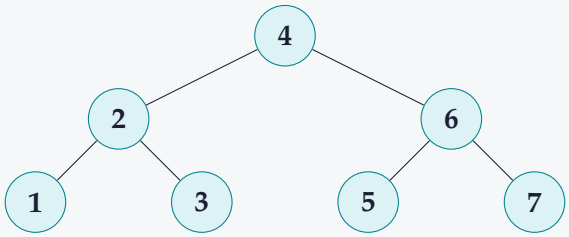
4.2 Binary Search Technique

Binary search operates on a **monotonically sorted** contiguous sequence via divide-and-conquer, halving the search space in each iteration.

Binary Search Algorithm (Iterative Implementation)

Algorithm BinarySearch(A, n, K)**:**
1. Set low $= 0$, high $= n - 1$.
2. While low $\leq$ high do:
3. Compute mid $=$ low $+ \lfloor(\text{high} - \text{low})/2\rfloor$. (*Prevents integer overflow*)
4. If $A[\text{mid}] == K$, return mid (Success).
5. Else if $A[\text{mid}] < K$, set low $=$ mid $+ 1$.
6. Else set high $=$ mid $- 1$.
7. Return -1 (Key Absent).

Decision Tree Representation of Binary Search on $n = 7$ Elements



At each stage, one comparison reduces the remaining search tree depth by 1. Total depth is $\lfloor \log_2 n \rfloor + 1$.

Binary Search Recurrence Relation and Complexity Derivation

In each step, the algorithm performs $\mathcal{O}(1)$ comparisons and reduces problem size n to $n/2$:

$$T(n) = T\left(\frac{n}{2}\right) + c, \qquad T(1) = 1 \quad (14)$$

Solving via Repeated Substitution (Unfolding):

$$T(n) = T\left(\frac{n}{2}\right) + c = \left[T\left(\frac{n}{4}\right) + c\right] + c = T\left(\frac{n}{2^2}\right) + 2c$$
$$= T\left(\frac{n}{2^k}\right) + k \cdot c$$

The process terminates when the subproblem reaches size 1: $\frac{n}{2^k} = 1 \implies 2^k = n \implies k = \log_2 n$.

$$T(n) = T(1) + c \cdot \log_2 n = 1 + c \log_2 n \implies \square(\log_2 \mathbf{n}) \quad (15)$$

© Biraj

Summary of Bounds:
- **Best-Case:** $A[\text{mid}] == K$ on first comparison $\implies \mathcal{O}(\mathbf{1})$.
- **Worst-Case:** Element is at the leaf of the decision tree or absent $\implies \mathcal{O}(\log_2 \mathbf{n})$.
- **Average-Case:** Successful search across all decision tree nodes $\implies \mathcal{O}(\log_2 \mathbf{n})$.
- **Space Complexity:** $\mathcal{O}(1)$ for iterative version; $\mathcal{O}(\log_2 n)$ stack frames for recursive version.

4.3 Comparison: Linear Search vs Binary Search

Criterion	Linear Search	Binary Search
Prerequisite	Elements can be arbitrary/unsorted	Sequence **must** be monotonically sorted
Access Requirement	Sequential or Random access (Array or Linked List)	Random access required (Arrays only; inefficient on linked lists)
Best-Case Time	$\mathcal{O}(1)$ (first item matches)	$\mathcal{O}(1)$ (middle item matches)
Worst-Case Time	$\mathcal{O}(n)$	$\mathcal{O}(\log_2 n)$
Average-Case Time	$\mathcal{O}(n)$	$\mathcal{O}(\log_2 n)$
Space Complexity	$\mathcal{O}(1)$ in-place	$\mathcal{O}(1)$ iterative, $\mathcal{O}(\log n)$ recursive
Hardware Efficiency	Excellent CPU cache spatial locality	Multiple cache misses on large data jumps
Algorithm Paradigm	Brute-force linear scan	Divide-and-Conquer

© Biraj

Quick Revision — Module 1

Introduction, Algorithm Analysis and Searching Techniques | ES-CS301

Key Formulas and Asymptotic Relations at a Glance

1D Array Address	$\text{Loc}(A[i]) = B + W \cdot (i - LB)$
2D Row-Major Address	$\text{Loc}(A[i][j]) = B + W \cdot [(i - LB_1) \cdot N_2 + (j - LB_2)]$
2D Column-Major Address	$\text{Loc}(A[i][j]) = B + W \cdot [(j - LB_2) \cdot N_1 + (i - LB_1)]$
Asymptotic Bounds $(\mathcal{O}, \Omega, \Theta)$	$f(n) \leq c_1 g(n)$ $(\mathcal{O})$, $c_2 g(n) \leq f(n)$ (Ω), $c_3 g(n) \leq f(n) \leq c_4 g(n)$ (Θ)
Strict Limits (o, ω)	$\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$ (o), $\lim_{n\to\infty} \frac{f(n)}{g(n)} = \infty$ (ω)
Binary Search Recurrence	$T(n) = T(n/2) + \Theta(1) \implies T(n) = \Theta(\log_2 n)$
Linear Search Bounds	Best-Case: $\mathcal{O}(1)$, Worst-Case / Average: $\mathcal{O}(n)$
Complexity Hierarchy	$\mathcal{O}(1) < \mathcal{O}(\log n) < \mathcal{O}(n) < \mathcal{O}(n \log n) < \mathcal{O}(n^2) < \mathcal{O}(2^n) < \mathcal{O}(n!)$

One-Line Definitions

- **Data Structure:** A logical and mathematical organization of data elements along with designated legal operations.
- **Abstract Data Type (ADT):** A mathematical model for data types defining behavior from user perspective without implementation details.
- **Linear Data Structure:** A data structure where elements form an ordered sequential sequence with 1-to-1 adjacent relations.
- **Row-Major Order (RMO):** A 2D mapping layout where all elements of row i precede row $i + 1$ in contiguous physical memory.
- **Column-Major Order (CMO):** A 2D mapping layout where all elements of column j precede column $j + 1$ in contiguous physical memory.
- **Big-O ($\mathcal{O}$):** An asymptotic notation defining a formal mathematical upper bound on algorithm computational growth.
- **Time-Space Trade-off:** The engineering principle where computational runtime is decreased by allocating extra memory.
- **Binary Search:** A divide-and-conquer algorithm on sorted arrays halving search space in $\mathcal{O}(\log_2 n)$ comparisons.

© Biraj

Frequently Asked / Viva Questions (Part 1)

Q1. What is the difference between a Data Type and a Data Structure?

A data type is a primitive classification defining a set of values and allowable low-level hardware operations (e.g., `int`, `float`). A data structure is a higher-level collection of data elements organized logically with specific algorithms to store and manipulate them efficiently in memory (e.g., arrays, stacks, trees).

Q2. Why do we prefer asymptotic notations over clock time for analyzing algorithms?

Physical clock time depends on machine architecture, processor speed, compiler optimizations, background processes, and cache memory. Asymptotic notations evaluate performance purely as a function of input size n, independent of hardware, providing an objective, universal measure of scalability.

Q3. State the necessary and sufficient condition for $f(n) = \Theta(g(n))$.

The function $f(n) = \Theta(g(n))$ if and only if $f(n) = \mathcal{O}(g(n))$ and $f(n) = \Omega(g(n))$. That is, $g(n)$ serves simultaneously as both the asymptotic upper bound and lower bound within positive constant multipliers c_1 and c_2.

Q4. Why must an array be sorted prior to executing Binary Search?

Binary search relies on the monotonicity of elements. When the target key K is compared with the middle element $A[\text{mid}]$, the algorithm assumes that all elements to the left are $\leq A[\text{mid}]$ and all elements to the right are $\geq A[\text{mid}]$. Without sorted order, discarding half the array could eliminate the target key.

Q5. Why is Binary Search inefficient when applied to a Singly Linked List?

Binary search requires finding the midpoint in $\mathcal{O}(1)$ time. In a linked list, random access is not supported ($A[i]$ takes $\mathcal{O}(i)$ traversal steps). Locating the middle node takes $\mathcal{O}(n)$ time, which negates the divide-and-conquer speedup, resulting in overall $\mathcal{O}(n)$ complexity.

Q6. Explain the concept of integer overflow in computing the midpoint in Binary Search and how it is resolved.

Writing `mid = (low + high) / 2` can cause integer overflow if the sum `low + high` exceeds the maximum 32-bit signed integer value ($2^{31} - 1$). The overflow wraps to a negative number, causing an index out-of-bounds fault. It is avoided using `mid = low + (high - low) / 2`.

Q7. Differentiate between Big-O ($\mathcal{O}$) and Little-o (o).

Big-O establishes an asymptotic upper bound that may be tight ($2n^2 = \mathcal{O}(n^2)$ is valid). Little-o establishes a strict, non-tight upper bound such that $f(n)$ grows strictly slower than $g(n)$ as $n \to \infty$ ($\lim_{n\to\infty} f(n)/g(n) = 0$). Thus $2n = o(n^2)$, but $2n^2 \neq o(n^2)$.

© Biraj

Frequently Asked / Viva Questions (Part 2)

Q8. What is the difference between Space Complexity and Auxiliary Space?

Space complexity is the total memory used by an algorithm, including both the input data and any extra working memory. Auxiliary space refers exclusively to the extra temporary or scratchpad memory allocated by the algorithm beyond the input size.

Q9. Provide an example of an engineering Time-Space trade-off.

In calculating Fibonacci numbers: naive recursion takes $\mathcal{O}(2^n)$ time and $\mathcal{O}(n)$ call-stack space. Using dynamic programming memoization with an auxiliary array of size n, runtime drops dramatically to $\mathcal{O}(n)$ time at the expense of $\mathcal{O}(n)$ extra storage space.

Q10. What are the worst-case, best-case, and average-case scenarios for Linear Search?

Best case is $\mathcal{O}(1)$ (element found at index 0). Worst case is $\mathcal{O}(n)$ (element is at index $n-1$ or absent). Average case is $\mathcal{O}(n)$ (specifically $(n+1)/2$ comparisons under a uniform random distribution).

Summary Matrix: Comprehensive Comparison of Searching and Analysis Fundamentals

Topic / Technique	Core Governing Relation	Complexity Bound	Key Engineering Takeaway
Row-Major 2D Addressing	**Loc** $= B + W[(i - LB_1)N_2 + (j - LB_2)]$	$\mathcal{O}(1)$ **compute time**	C/C++ memory layout; row iteration preserves cache locality.
Column-Major 2D Addressing	**Loc** $= B + W[(j - LB_2)N_1 + (i - LB_1)]$	$\mathcal{O}(1)$ **compute time**	Fortran/MATLAB layout; column-by-column iteration preferred.
Big-O Notation ($\mathcal{O}$)	$f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$	**Upper bound**	Worst-case computational ceiling guarantees.
Big-Theta Notation (Θ)	$c_1 g(n) \leq f(n) \leq c_2 g(n)$	**Tight bound**	Exact algorithmic rate of growth.
Linear Search	**Sequential traversal** $A[0 \ldots n-1]$	**Worst:** $\mathcal{O}(n)$, **Best:** $\mathcal{O}(1)$	Unsorted or small lists; works natively on linked lists.
Binary Search	$T(n) = T(n/2) + \mathcal{O}(1)$	**Worst:** $\mathcal{O}(\log n)$, **Best:** $\mathcal{O}(1)$	Strictly requires sorted array and random access memory.

© Biraj

Module 2

Abstract Data Type Stack and Fundamental Operations

1. Abstract Data Type Stack and Fundamental Operations

Definition: Stack Abstract Data Type (ADT)

A **Stack** is a linear, restricted-access data structure adhering strictly to the **Last-In, First-Out (LIFO)** or **First-In, Last-Out (FILO)** operating discipline. All insertions (`push`) and deletions (`pop`) are constrained to take place at a single end termed the **Top of Stack (TOS)**.

Formally, a Stack S over an element domain E is defined by:

- State: An ordered sequence $S = \langle e_0, e_1, \dots, e_k \rangle$ where e_k is the Top element.
- Auxiliary Pointer: An integer index top $\in [-1, \text{MAX} - 1]$ indicating the index of the most recently inserted item.

1.1 Array Implementation of Stack

In an array-based static implementation, elements reside in contiguous storage $S[0 \dots \text{MAX} - 1]$.

- **Initial Condition:** top $= -1$ (stack is strictly empty).
- **Stack Overflow Condition:** Attempting to insert an element when the physical capacity is exhausted:

$$\text{Overflow} \iff \text{top} == \text{MAX} - 1 \tag{16}$$

- **Stack Underflow Condition:** Attempting to retrieve or delete an element from an empty stack:

$$\text{Underflow} \iff \text{top} == -1 \tag{17}$$

Core Stack Primitive Algorithms and Complexity Bounds

1. Algorithm Push(S, top, x):

```
if (top == MAX - 1)
    report "Stack Overflow" and exit error;
top = top + 1;
S[top] = x;
```

Time Complexity: $\mathcal{O}(\mathbf{1})$ Space Complexity: $\mathcal{O}(\mathbf{1})$

2. Algorithm Pop(S, top):

© Biraj

```
if (top == -1)
    report "Stack Underflow" and exit error;
item = S[top];
top = top - 1;
return item;
```

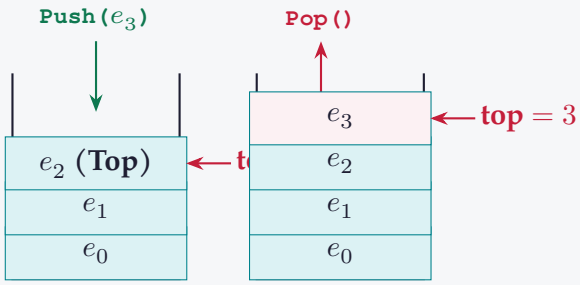
Time Complexity: $\mathcal{O}(\mathbf{1})$ Space Complexity: $\mathcal{O}(\mathbf{1})$

3. Algorithm Peek(S, top):

```
if (top == -1)
    report "Stack Empty" and exit error;
return S[top]; // Non-destructive inspection
```

Time Complexity: $\mathcal{O}(\mathbf{1})$ Space Complexity: $\mathcal{O}(\mathbf{1})$

LIFO Mechanism and State Transitions in Stack Operations



2. Applications of Stacks

2.1 Arithmetic Expression Formats and Operator Precedence

Mathematical expressions are represented in three distinct syntactic notations:

1. **Infix Notation:** Binary operators are placed between operands: ⟨Operand⟩ **Op** ⟨Operand⟩ (e.g., $(A + B) * C$). Requires explicit parentheses and precedence parsing rules.
2. **Postfix Notation (Reverse Polish Notation — RPN):** Operators appear immediately following their operands: ⟨Operand⟩ ⟨Operand⟩ **Op** (e.g., $AB + C*$). Evaluated linearly without parentheses or backtrack searching.
3. **Prefix Notation (Polish Notation):** Operators precede operands: **Op** ⟨Operand⟩ ⟨Operand⟩ (e.g., $* + ABC$).

© Biraj

Operator	Operation Description	Associativity	Precedence Rank
`^` or `$`	**Exponentiation**	**Right-to-Left (R → L)**	Highest (Rank 3)
`*`, `/`, `%`	**Multiplication, Division, Modulo**	**Left-to-Right (L → R)**	Intermediate (Rank 2)
`+`, `-`	**Addition, Subtraction**	**Left-to-Right (L → R)**	Lowest (Rank 1)
`(`	**Left Parenthesis**	**Left-to-Right**	Rank 0 (Inside Stack)

2.2 Infix to Postfix Conversion Algorithm

Shunting-Yard Infix to Postfix Algorithm

1. Initialize an empty operator stack S and an empty postfix output string P.
2. Scan the infix expression string I from left to right, character by character:
   - **If Operand:** Append directly to postfix string P.
   - **If Left Parenthesis `'('`:** Push onto stack S.
   - **If Right Parenthesis `')'`:** Pop from S and append to P repeatedly until `'('` is popped. Discard both parentheses.
   - **If Operator** op**:** While stack is non-empty, and Prec(top) $>$ Prec(op) (or Prec(top) $==$ Prec(op) with Left-Associativity), pop S and append to P. Then push op onto S.
3. After end of input string, pop all remaining operators from S and append to P.

Step-by-Step Trace: Infix to Postfix Conversion

Convert: $I = A + B * (C \wedge D - E) \wedge (F + G * H) - I$

© Biraj

Token	Operator Stack	Postfix Output P	Action
A	$\emptyset$	A	**Append operand**
$+$	$+$	A	**Push operator**
B	$+$	AB	**Append operand**
$*$	$+\ *$	AB	**Push (higher precedence)**
$($	$+\ *\ ($	AB	**Push parenthesis**
C	$+\ *\ ($	ABC	**Append operand**
$\wedge$	$+\ *\ (\ \wedge$	ABC	**Push exponent**
D	$+\ *\ (\ \wedge$	$ABCD$	**Append operand**
$-$	$+\ *\ (\ -$	$ABCD\wedge$	**Pop $\wedge$, push $-$**
E	$+\ *\ (\ -$	$ABCD\wedge E$	**Append operand**
$)$	$+\ *$	$ABCD\wedge E-$	**Pop until $($, discard**
$\wedge$	$+\ *\ \wedge$	$ABCD\wedge E-$	**Push (higher precedence)**
$($	$+\ *\ \wedge\ ($	$ABCD\wedge E-$	**Push parenthesis**
F	$+\ *\ \wedge\ ($	$ABCD\wedge E-F$	**Append operand**
$+$	$+\ *\ \wedge\ (\ +$	$ABCD\wedge E-F$	**Push operator**
G	$+\ *\ \wedge\ (\ +$	$ABCD\wedge E-FG$	**Append operand**
$*$	$+\ *\ \wedge\ (\ +\ *$	$ABCD\wedge E-FG$	**Push (higher precedence)**
H	$+\ *\ \wedge\ (\ +\ *$	$ABCD\wedge E-FGH$	**Append operand**
$)$	$+\ *\ \wedge$	$ABCD\wedge E-FGH*+$	**Pop until $($, discard**
$-$	$-$	$ABCD\wedge E-FGH*+\wedge *+$	**Pop $\wedge, *, +$, push $-$**
I	$-$	$ABCD\wedge E-FGH*+\wedge *+I$	**Append operand**
End	$\emptyset$	**ABCD $\wedge$ E $-$ FGH $*+\wedge *+$ I$-$**	**Pop all remaining**

2.3 Evaluation of Postfix Expressions

Postfix Expression Evaluation Algorithm

Scan postfix string P from left to right:

1. If current symbol is an operand, push its numerical value onto operand stack S.
2. If symbol is an operator $\odot$:
   - $\text{op}_2 = \text{Pop}(S)$ (*Second operand popped first!*)
   - $\text{op}_1 = \text{Pop}(S)$ (*First operand popped second!*)
   - Compute result $= \text{op}_1 \odot \text{op}_2$.
   - $\text{Push}(S, \text{result})$.
3. When string ends, the final evaluated result resides at $\text{Pop}(S)$.

© Biraj

Time Complexity: $\mathcal{O}(\mathbf{n})$ Auxiliary Space: $\mathcal{O}(\mathbf{n})$

Evaluation Trace: Numerical Postfix String

Evaluate: $P = 6\ 2\ 3\ +\ *\ 12\ 4\ /\ -$

- Scan `6`, `2`, `3`: Push to stack $\implies S = [6, 2, 3]$.
- Encounter `+`: Pop $\text{op}_2 = 3$, $\text{op}_1 = 2$. Compute $2 + 3 = 5$. Push 5 $\implies S = [6, 5]$.
- Encounter `*`: Pop $\text{op}_2 = 5$, $\text{op}_1 = 6$. Compute $6 * 5 = 30$. Push 30 $\implies S = [30]$.
- Scan `12`, `4`: Push to stack $\implies S = [30, 12, 4]$.
- Encounter `/`: Pop $\text{op}_2 = 4$, $\text{op}_1 = 12$. Compute $12/4 = 3$. Push 3 $\implies S = [30, 3]$.
- Encounter `-`: Pop $\text{op}_2 = 3$, $\text{op}_1 = 30$. Compute $30 - 3 = 27$. Push 27 $\implies S = [27]$.
- End of expression: Final Result = **27**.

2.4 Infix to Prefix Conversion and Prefix Evaluation

Infix to Prefix Conversion Algorithm (Reversal Method)

To convert an infix expression I to prefix notation Pre:
1. Reverse the infix string I.
2. In the reversed string, replace every `'('` with `')'` and every `')'` with `'('`.
3. Compute the postfix expression of this transformed string using the Shunting-Yard algorithm (with operator precedence rules: if precedence of scanned operator equals TOS, operators with right-associativity pop TOS, left-associative operators are pushed).
4. Reverse the resulting postfix string. The final string is the valid **Prefix expression**.

Time Complexity: $\mathcal{O}(\mathbf{n})$ Auxiliary Space: $\mathcal{O}(\mathbf{n})$

Example: Infix to Prefix Conversion

Convert: $I = (A + B) * C - (D - E) * (F + G)$
1. Reverse I: $(G + F) * (E - D) - C * (B + A)$
2. Swap brackets: $(G + F) * (E - D) - C * (B + A)$
3. Postfix of transformed string: $GF + ED - *CBA + *-$
4. Reverse result: $- * +$**ABC** $* -$**DE** $+$ **FG** (or $- * +ABC * -DE + FG$)

Prefix Expression Evaluation Algorithm

Scan the prefix expression Pre from **Right to Left**:
1. If current symbol is an operand, push its value onto operand stack S.
2. If symbol is an operator $\odot$:
   - $\text{op}_1 = \text{Pop}(S)$ (*First operand popped first!*)
   - $\text{op}_2 = \text{Pop}(S)$ (*Second operand popped second!*)
   - Compute result $= \text{op}_1 \odot \text{op}_2$.
   - Push(S, result).

© Biraj

3. When scan completes, Pop(S) contains the final evaluated scalar value.

2.5 Balanced Delimiter Checking and Call Stack Simulation

Algorithm: Delimiter Matching (Parentheses Balancing)

Algorithm CheckBalanced$(expr)$**:**
1. Initialize an empty character stack S.
2. For each character ch in string $expr$:
   - If $ch \in \{$`'('`, `'['`, `'{'`$\}$, execute Push(S, ch).
   - If $ch \in \{$`')'`, `']'`, `'}'`$\}$:
     - If IsEmpty(S), return **False** (Underflow: unmatched closing bracket).
     - $topChar =$ Pop(S).
     - If $(topChar, ch)$ is not a matching pair `()`, `[]`, or `{}`, return **False**.
3. Return IsEmpty(S) (True if all opened brackets were closed; False otherwise).

Time Complexity: $\mathcal{O}(\mathbf{n})$ Space Complexity: $\mathcal{O}(\mathbf{n})$

Function Call Stack and Recursion Architecture

When a recursive function (such as $n!$ or Fibonacci) executes, the operating system instantiates an **Activation Record (Stack Frame)** on the thread's call stack for each recursive call:
- Contents of a Frame: Return Address, Function Parameters, Local Variables, Saved Base Pointer (`EBP/RBP`).
- Growth: As calls nest deeper, frames stack vertically. Winding phase consumes $\mathcal{O}(n)$ stack space.
- Unwinding: Upon reaching base case, frames pop in LIFO sequence, returning values to caller.

3. Abstract Data Type Queue and Linear Queue

Definition: Queue Abstract Data Type (ADT)

A **Queue** is a linear, homogeneous data structure governed by the **First-In, First-Out (FIFO)** operating principle. Insertions are restricted to the **Rear** terminal, while deletions occur exclusively at the **Front** terminal.

Pointers in an array of size MAX:
- front: Points to the position of the first valid data element (or one position prior in alternate textbook conventions).
- rear: Points to the index of the latest inserted data item.

3.1 Array Implementation of Linear Queue

Let array storage be $Q[0 \ldots \text{MAX} - 1]$ with standard 0-based indices:

- **Initial Empty State:** front $= -1$, rear $= -1$.

© Biraj

- **Queue Underflow Condition:** front $== -1$ (or front $>$ rear).
- **Queue Overflow Condition:** rear $==$ MAX $- 1$.

Linear Queue Algorithms and the False Overflow Anomaly

1. Algorithm Enqueue(Q, x):

```
if (rear == MAX - 1)
    report "Queue Overflow" and exit error;
if (front == -1) front = 0; // First element insertion
rear = rear + 1;
Q[rear] = x;
```

2. Algorithm Dequeue(Q):

```
if (front == -1 || front > rear)
    report "Queue Underflow" and exit error;
item = Q[front];
front = front + 1;
if (front > rear) // Reset queue when emptied
    front = -1, rear = -1;
return item;
```

The False Overflow Problem (Memory Drift): In a simple linear queue, after several enqueue and dequeue cycles, rear reaches MAX $- 1$ while front has advanced forward (e.g., front $= 4$, rear $=$ MAX $- 1$). Even though indices $0 \ldots 3$ are physically empty, the condition rear $==$ MAX $- 1$ reports overflow and rejects new insertions!

4. Advanced Queue Architectures

4.1 Circular Queue

A **Circular Queue** overcomes false overflow by logically connecting index MAX $- 1$ back to index 0 via **modulo arithmetic**:

$$\text{next}(\text{index}) = (\text{index} + 1) \bmod \text{MAX} \tag{18}$$

Circular Queue Boundary States and Algorithms

Let $Q[0 \ldots \text{MAX} - 1]$:

- **Initial Empty State:** front $= -1,$ rear $= -1$.
- **Circular Queue Full Condition:**

$$(\text{rear} + 1) \bmod \text{MAX} == \text{front} \tag{19}$$

- **Circular Queue Empty Condition:**

$$\text{front} == -1 \tag{20}$$

© Biraj

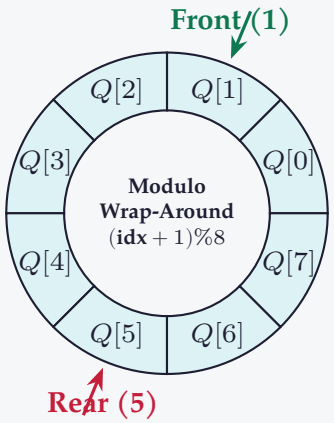
Algorithm CircularEnqueue(Q, x):

```
if ((rear + 1) % MAX == front)
    report "Circular Queue Overflow" and exit;
if (front == -1) // First element
    front = 0, rear = 0;
else
    rear = (rear + 1) % MAX;
Q[rear] = x;
```

Algorithm CircularDequeue(Q):

```
if (front == -1)
    report "Circular Queue Underflow" and exit;
item = Q[front];
if (front == rear) // Only one element was present
    front = -1, rear = -1;
else
    front = (front + 1) % MAX;
return item;
```

Ring Representation of Circular Queue ($MAX = 8$)



4.2 Priority Queue

A **Priority Queue** is an ADT where each element has an associated priority key. Elements are processed according to priority rather than arrival time:

- **Ascending Priority Queue:** Item with the lowest numerical priority value is extracted first.
- **Descending Priority Queue:** Item with the highest numerical priority value is extracted first.

© Biraj

Implementation	Enqueue Time	Dequeue Time	Trade-off Remarks
Unordered Array	$\mathcal{O}(1)$	$\mathcal{O}(n)$	Linear search required to find extreme priority.
Ordered Array	$\mathcal{O}(n)$	$\mathcal{O}(1)$	Sorted insertion requires element shifts.
Unordered Linked List	$\mathcal{O}(1)$	$\mathcal{O}(n)$	Deletion requires traversing entire list.
Ordered Linked List	$\mathcal{O}(n)$	$\mathcal{O}(1)$	Dequeue is $\mathcal{O}(1)$ removal at head.
Binary Heap	$\mathcal{O}(\log \mathbf{n})$	$\mathcal{O}(\log \mathbf{n})$	**Optimal engineering balance** (standard in C++/Java).

4.3 Double-Ended Queue (DEQUE)

A **DEQUE** (Double-Ended Queue) permits insertion and deletion at both the front and rear ends:

1. **Input-Restricted DEQUE:** Insertion is allowed at only one end (e.g., rear), while deletion is allowed at both ends (front and rear).
2. **Output-Restricted DEQUE:** Deletion is allowed at only one end (e.g., front), while insertion is allowed at both ends (front and rear).

© Biraj

Quick Revision — Module 2

Stacks and Queues: ADTs, Operations and Applications | ES-CS301

Key Formulas and Boundary Conditions at a Glance

Stack Overflow / Underflow	Overflow: top $==$ MAX -1, Underflow: top $==-1$
Linear Queue Full / Empty	Full: rear $==$ MAX -1, Empty: front $==-1$ or front $>$ rear
Circular Queue Next Pointer	next $=$ (index $+1$) mod MAX
Circular Queue Full Condition	(rear $+1$) mod MAX $==$ front
Circular Queue Empty Condition	front $==-1$
Circular Queue Element Count	Count $=$ (rear $-$ front $+$ MAX) mod MAX $+1$ (if non-empty)
Stack Operation Complexities	Push: $\mathcal{O}(1)$, Pop: $\mathcal{O}(1)$, Peek: $\mathcal{O}(1)$
Binary Heap Priority Queue	Insert: $\mathcal{O}(\log n)$, Extract-Min/Max: $\mathcal{O}(\log n)$

One-Line Definitions

- **Stack:** A linear data structure operating under the Last-In, First-Out (LIFO) discipline at a single end.
- **Queue:** A linear data structure operating under the First-In, First-Out (FIFO) discipline (front/rear).
- **False Overflow:** Rejection of enqueue requests in a linear queue when rear $==$ MAX -1 despite empty slots at front.
- **Circular Queue:** A fixed-size queue that wraps around using modulo arithmetic to prevent memory drift.
- **Priority Queue:** An ADT where elements are retrieved based on an assigned priority key rather than FIFO arrival.
- **DEQUE:** A double-ended queue permitting insertion and deletion operations at both extremities.
- **Postfix Notation (RPN):** Parenthesis-free operator-after-operands notation evaluated in a single linear pass via a stack.
- **Activation Record:** A stack frame containing subroutine parameters, local variables, and return address.

© Biraj

Frequently Asked / Viva Questions (Part 1)

Q1. What is the primary difference between LIFO and FIFO operating principles?

LIFO (Last-In, First-Out), implemented by Stacks, ensures that the most recently inserted item is the first to be retrieved. FIFO (First-In, First-Out), implemented by Queues, guarantees that elements are serviced strictly in chronological order of arrival.

Q2. Why are Postfix expressions preferred over Infix expressions by compiler evaluators?

Infix expressions require operator precedence tables, associativity rules, and parenthetical parsing, necessitating lookaheads and recursive evaluation. Postfix expressions are completely parenthesis-free and can be evaluated in a single linear pass ($\mathcal{O}(n)$ time) using an operand stack.

Q3. How does a Circular Queue solve the False Overflow issue of a Linear Queue?

In a linear queue, deleted slots before `front` cannot be reused once `rear` reaches $\text{MAX} - 1$. A circular queue treats the array as a closed ring using modulo arithmetic ($(\text{rear} + 1) \bmod \text{MAX}$), wrapping new insertions into vacant slots at the beginning of the buffer.

Q4. During postfix evaluation, why must the first popped element be assigned as the second operand?

Because stack operations are LIFO. If $A-B$ is converted to $AB-$, B is pushed after A and is therefore on top. Popping retrieves B first (op_2) and A second (op_1). For non-commutative operators (subtraction, division), swapping operands yields incorrect results ($\text{op}_1 - \text{op}_2 \neq \text{op}_2 - \text{op}_1$).

Q5. State the condition for a Circular Queue of size MAX to be completely full.

The standard condition is $(\text{rear} + 1) \bmod \text{MAX} == \text{front}$. This ensures that advancing the rear pointer by one slot would collide with the current front position.

Q6. What is the difference between an Input-Restricted and an Output-Restricted DEQUE?

In an Input-Restricted DEQUE, insertions are permitted at only one end (e.g., rear), while deletions can occur at both ends (front and rear). In an Output-Restricted DEQUE, deletions are restricted to one end (e.g., front), while insertions can take place at both ends.

Q7. How can a stack be used to check for balanced parentheses in an expression?

Traverse the expression character by character. When an opening delimiter (`(`, `[`, `{`) is found, push it onto the stack. When a closing delimiter (`)`, `]`, `}`) is found, pop the stack and check if it matches the closing symbol. If underflow occurs or types mismatch, it is unbalanced. The expression is balanced if the stack is empty at the end.

© Biraj

Frequently Asked / Viva Questions (Part 2)

Q8. Why is a Binary Heap the preferred data structure for implementing a Priority Queue?

In an array or linked list, either insertion or extraction is $\mathcal{O}(n)$. A Binary Heap achieves an optimal compromise of $\mathcal{O}(\log n)$ for both insertion and extraction of the extreme-priority item, making it ideal for dynamic workloads.

Q9. What happens to the call stack during infinite recursion, and what error is generated?

Each recursive call allocates an activation frame on the runtime call stack. Without a valid terminating base condition, stack frames accumulate until memory allocated for the call stack is exhausted, triggering a fatal **Stack Overflow** error.

Q10. Can two stacks be implemented efficiently within a single one-dimensional array?

Yes. Stack 1 starts at index 0 and grows rightward ($\text{top}_1 ++$), while Stack 2 starts at index MAX $-$ 1 and grows leftward ($\text{top}_2 --$). Overflow occurs only when $\text{top}_1 + 1 == \text{top}_2$, maximizing total array space utilization.

Summary Matrix: Comprehensive Comparison of Stack and Queue Structures

Structure	Discipline / Pointers	Critical Conditions	Primary Practical Applications
Stack	**LIFO / FILO; Single pointer top**	**Full:** **top** $==$ **MAX** -1 **Empty:** **top** $==-1$	Expression parsing, recursion, backtracking, undo operations.
Linear Queue	**FIFO / LILO; Two pointers front, rear**	**Full:** **rear** $==$ **MAX** -1 **Empty:** **front** $==-1$	Print spooling, simple task buffering (suffers memory drift).
Circular Queue	**Ring FIFO; Modulo wrap-around**	**Full:** (**rear** $+$ $1)\%$**MAX** $==$ **front** **Empty:** **front** $==-1$	CPU scheduling (round robin), IO buffering, audio streaming.
Priority Queue	**Priority-ordered; Key-based selection**	**Governed by underlying heap/array capacity**	Dijkstra's algorithm, Huffman coding, OS interrupt processing.
DEQUE	**Double-ended; Front & Rear access**	**Full: depends on circular or linear array implementation**	Browser tab history, job-stealing algorithms, palindrome verification.

© Biraj

Module 3

Linked Lists Fundamentals and Singly Linked List (SLL)

1. Linked Lists Fundamentals and Singly Linked List (SLL)

Definition: Singly Linked List (SLL) Architecture

A **Singly Linked List** is a dynamic linear data structure composed of distinct nodes allocated at arbitrary non-contiguous physical memory locations. Each node consists of two essential components:

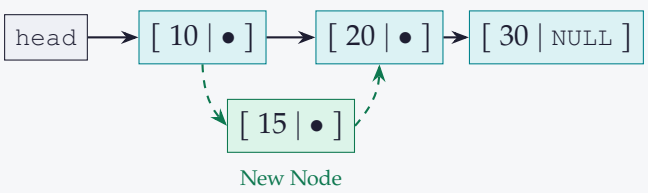
1. **Data Field:** Stores the actual information payload of type T.
2. **Link (Next Pointer):** Stores the explicit physical memory address of the logically succeeding node.

The list is anchored by an external pointer `head` referencing the first node. The terminal node's link contains `NULL`.

1.1 Limitations of Arrays vs Advantages of Linked Lists

- **Memory Allocation:** Arrays require fixed, compile-time contiguous memory allocation. Linked lists allocate memory dynamically on the heap at runtime via `malloc()` / `free()`, growing and shrinking on demand.
- **Insertion and Deletion Overhead:** Inserting or deleting an element at index 0 in an array of size n requires shifting all n subsequent elements ($\mathcal{O}(n)$ time). In a linked list, inserting or deleting at the head requires updating pointer references, which executes in strictly $\mathcal{O}(\mathbf{1})$ time without moving physical payloads.
- **Memory Fragmentation:** Arrays can suffer allocation failure due to external memory fragmentation even when total free RAM exceeds the array size; linked lists utilize scattered memory chunks effectively.

Memory Representation and Pointer Rewiring in a Singly Linked List



© Biraj

1.2 Singly Linked List Algorithmic Operations

Algorithms for Insertion and Deletion in SLL

1. Insertion at Beginning ($\mathcal{O}(1)$):

```
newNode = allocateNode(val);
newNode->next = head;
head = newNode;
```

2. Insertion at End ($\mathcal{O}(n)$ without tail pointer, $\mathcal{O}(1)$ with tail):

```
newNode = allocateNode(val); newNode->next = NULL;
if (head == NULL) { head = newNode; return; }
ptr = head;
while (ptr->next != NULL) ptr = ptr->next;
ptr->next = newNode;
```

3. Deletion at Beginning ($\mathcal{O}(1)$):

```
if (head == NULL) report "Underflow";
temp = head;
head = head->next;
free(temp);
```

4. Deletion of a Node with Value X ($\mathcal{O}(n)$):

```
if (head == NULL) return;
if (head->data == X) { temp = head; head = head->next; free(temp); return; }
curr = head; prev = NULL;
while (curr != NULL && curr->data != X) { prev = curr; curr = curr->next; }
if (curr == NULL) report "Key Not Found";
else { prev->next = curr->next; free(curr); }
```

2. Linked Representation of Stack and Queue & Header Nodes

2.1 Linked Stack and Linked Queue

- **Linked Stack:** Implemented by inserting and deleting strictly at the head node of the linked list:
  - Push(x) $\equiv$ Insert at Beginning $\implies$ $\mathcal{O}(\mathbf{1})$
  - Pop() $\equiv$ Delete from Beginning $\implies$ $\mathcal{O}(\mathbf{1})$
  - Overflow never occurs unless the system runs out of heap memory.
- **Linked Queue:** Maintained via two external pointers: `front` and `rear`:
  - Enqueue(x) $\equiv$ Insert at End (via `rear->next` = newNode; `rear` = newNode) $\implies$ $\mathcal{O}(\mathbf{1})$
  - Dequeue() $\equiv$ Delete at Head (via `front` = `front->next`) $\implies$ $\mathcal{O}(\mathbf{1})$

© Biraj

2.2 Header Nodes in Linked Lists

A **Header Node** is a dummy node placed at the very beginning of the linked list before the first actual data node:

- **Grounded Header List:** Header node's link points to the first data node; the last node's link contains `NULL`.
- **Circular Header List:** The last node's link points back to the header node itself.
- **Key Advantages:**
  1. Eliminates special-case coding for operations at the head (head pointer never needs re-assignment).
  2. The list is never truly empty (`head != NULL` always holds; an empty list has `head->next == NULL` or `head->next == head`).
  3. The header node's data field can store metadata (e.g., total node count, maximum element).

3. Doubly Linked Lists (DLL) and Circular Linked Lists (CLL)

3.1 Doubly Linked List (DLL)

In a **Doubly Linked List**, each node contains three fields: `prev` pointer, `data`, and `next` pointer.

- **Invariant:** `p->next->prev` == `p` and `p->prev->next` == `p`.
- **Bidirectional Traversal:** Allows forward and reverse scanning from any arbitrary node.
- $\mathcal{O}(1)$ **Deletion Given Node Pointer:** Deleting node P does not require searching for its predecessor:

```
P->prev->next = P->next;
if (P->next != NULL) P->next->prev = P->prev;
free(P);
```

- **Trade-off:** Extra memory overhead for backward pointers ($2\times$ pointer overhead per node) and additional pointer adjustments during insertions.

3.2 Circular Linked Lists (CLL)

In a **Circular Singly Linked List**, the last node's link points back to the first node.

- **Tail Pointer Paradigm:** By maintaining a single pointer `tail` to the last node:
  - Last node is accessible in $\mathcal{O}(1)$ (`tail`).
  - First node is accessible in $\mathcal{O}(1)$ (`tail->next`).
  - Both insertion at head and insertion at tail execute in strictly $\mathcal{O}(\mathbf{1})$ time!
- **Applications:** Continuous cycling processes, round-robin CPU scheduling, multiplayer turn games.

© Biraj

3.3 Polynomial Representation and Addition using Linked Lists

A single-variable polynomial $P(x) = a_n x^{e_n} + a_{n-1} x^{e_{n-1}} + \cdots + a_0 x^{e_0}$ is represented as a linked list where each node contains: `coeff` (coefficient), `exp` (exponent), and `next` pointer, maintained in strictly decreasing order of exponents.

Algorithm: Polynomial Addition using Singly Linked Lists

Algorithm AddPolynomials(P_1, P_2)**:**
1. Initialize pointers $p = P_1$, $q = P_2$, and an empty resultant list P_3.
2. While $p \neq$ NULL and $q \neq$ NULL:
   - **Case 1 ($p \to$ exp $== q \to$ exp):** Compute sum $= p \to$ coeff $+ q \to$ coeff. If sum $\neq 0$, insert term $(\text{sum}, p \to \text{exp})$ into P_3. Advance $p = p \to$ next, $q = q \to$ next.
   - **Case 2 ($p \to$ exp $> q \to$ exp):** Insert term $(p \to \text{coeff}, p \to \text{exp})$ into P_3. Advance $p = p \to$ next.
   - **Case 3 ($p \to$ exp $< q \to$ exp):** Insert term $(q \to \text{coeff}, q \to \text{exp})$ into P_3. Advance $q = q \to$ next.
3. Copy any remaining terms from p or q to the tail of P_3. Return P_3.

Time Complexity: $\mathcal{O}(\mathbf{m} + \mathbf{n})$ where m, n are the number of non-zero terms in P_1, P_2.

4. Tree Terminologies and Binary Tree Foundations

Definition: Tree Data Structure and Core Terminologies

A **Tree** is a non-linear, hierarchical data structure consisting of a collection of nodes connected by directed or undirected edges, such that there exists a unique root node R and no cycles.

- **Root:** The topmost ancestor node with in-degree 0 (no incoming parent edge).
- **Leaf (Terminal Node):** A node with out-degree 0 (no child nodes).
- **Degree of a Node:** The total number of subtrees or immediate children attached to that node.
- **Degree of a Tree:** The maximum degree among all nodes in the tree.
- **Level and Depth:** The root is at Level 0 (or 1). The depth of a node is the length of the path from the root to that node.
- **Height of a Tree (h):** The maximum depth among all nodes in the tree (length of the longest path from root to leaf).

4.1 Mathematical Properties of Binary Trees

A **Binary Tree** is a finite tree in which every node has at most two children, distinguished as **left child** and **right child**.

© Biraj

Core Mathematical Theorems on Binary Trees

1. Maximum number of nodes at level i (with root at level 0):

$$N_{\max}(i) = 2^i \tag{21}$$

2. Maximum total nodes in a binary tree of height h:

$$N_{\max}(h) = \sum_{i=0}^{h} 2^i = 2^{h+1} - 1 \tag{22}$$

3. Minimum height of a binary tree with n nodes:

$$h_{\min} = \lceil \log_2(n+1) \rceil - 1 \tag{23}$$

4. **Leaf vs Degree-2 Nodes Theorem:** In any non-empty binary tree, the number of leaf nodes n_0 is exactly one more than the number of nodes with degree two (n_2):

$$\mathbf{n_0 = n_2 + 1} \tag{24}$$

Proof: Let total nodes be $n = n_0 + n_1 + n_2$. Total edges in a tree of n nodes is $E = n - 1$. Also, each degree-1 node contributes 1 outgoing branch and each degree-2 node contributes 2 outgoing branches: $E = n_1 + 2n_2$. Equating the two expressions:

$$n_0 + n_1 + n_2 - 1 = n_1 + 2n_2 \implies n_0 - 1 = n_2 \implies \mathbf{n_0 = n_2 + 1}$$

4.2 Binary Tree Classifications

- **Full (Strict / Proper) Binary Tree:** Every internal node has exactly two children (0 or 2 children; no node has degree 1).
- **Complete Binary Tree:** All levels are completely filled except possibly the last level, where nodes are filled strictly from left to right without gaps. Foundation of array-based Binary Heaps.
- **Perfect Binary Tree:** All internal nodes have two children and all leaves reside at the same maximum depth (contains exactly $2^{h+1} - 1$ nodes).
- **Degenerate (Skewed) Binary Tree:** Every internal node has only one child, degrading to a linear linked list ($\mathcal{O}(n)$ height).

4.3 Binary Tree Traversals

1. **Preorder Traversal (D-L-R):** Process Data → Traverse Left subtree → Traverse Right subtree.
2. **Inorder Traversal (L-D-R):** Traverse Left subtree → Process Data → Traverse Right subtree. Produces sorted sequence for a BST!
3. **Postorder Traversal (L-R-D):** Traverse Left subtree → Traverse Right subtree → Process Data. Used for deleting tree bottom-up and expression evaluation.
4. **Level-Order Traversal (BFS):** Visit nodes level by level from top to bottom, left to right, implemented using a FIFO Queue.

© Biraj

5. Threaded Binary Tree

Definition: Threaded Binary Tree

In an n-node binary tree, exactly $2n$ pointer fields exist, of which $n-1$ point to children and $n+1$ are `NULL`. A **Threaded Binary Tree** utilizes these wasted null pointers:
- A `NULL` left pointer is converted into a **thread** pointing to the node's **Inorder Predecessor**.
- A `NULL` right pointer is converted into a **thread** pointing to the node's **Inorder Successor**.

Boolean tags (`lTag`, `rTag`) indicate whether the pointer field is a normal child link (0) or a thread (1).

Primary Advantage: Enables complete Inorder traversal without requiring recursive calls or an auxiliary stack, utilizing $\mathcal{O}(\mathbf{1})$ auxiliary memory space.

6. Binary Search Tree (BST)

Definition: Binary Search Tree (BST) Invariant

A **Binary Search Tree (BST)** is a binary tree in which every node x satisfies the BST ordering property:
- For all nodes y in the left subtree of x: $\text{Key}(y) < \text{Key}(x)$.
- For all nodes z in the right subtree of x: $\text{Key}(z) > \text{Key}(x)$.

Duplicate keys are either disallowed or handled uniformly in right subtrees.

6.1 BST Operations and Recursive Algorithms

Core Binary Search Tree Algorithms

1. Algorithm BST_Search($root, K$):

```
if (root == NULL || root->data == K) return root;
if (K < root->data) return BST_Search(root->left, K);
return BST_Search(root->right, K);
```

2. Algorithm BST_Insert($root, K$):

```
if (root == NULL) return createNode(K);
if (K < root->data) root->left = BST_Insert(root->left, K);
else if (K > root->data) root->right = BST_Insert(root->right, K);
return root;
```

3. Algorithm BST_Delete($root, K$):

```
if (root == NULL) return root;
if (K < root->data) root->left = BST_Delete(root->left, K);
```


© Biraj

```
else if (K > root->data) root->right = BST_Delete(root->right, K);
else {
    // Case 1: Leaf node or Case 2: One child
    if (root->left == NULL) {
        temp = root->right; free(root); return temp;
    } else if (root->right == NULL) {
        temp = root->left; free(root); return temp;
    }
    // Case 3: Two children
    temp = findMin(root->right); // Inorder Successor
    root->data = temp->data;
    root->right = BST_Delete(root->right, temp->data);
}
return root;
```

Complexity: Average $\mathcal{O}(\log_2 \mathbf{n})$, Worst (skewed tree) $\mathcal{O}(\mathbf{n})$.

7. AVL Tree (Self-Balancing Binary Search Tree)

Definition: AVL Tree and Balance Factor

An **AVL Tree** is a strictly height-balanced Binary Search Tree where the **Balance Factor** (BF) of every node v is constrained to $\{-1, 0, +1\}$:

$$BF(v) = h(\text{left subtree}) - h(\text{right subtree}) \in \{-1, 0, +1\} \tag{25}$$

If an insertion or deletion causes $|BF(v)| \geq 2$, the tree is rebalanced locally via rotations.

7.1 AVL Rebalancing Rotations

1. **LL Rotation (Single Right Rotation):** Applied when insertion occurs in the left subtree of the left child ($BF = +2$, child $BF = +1$).
2. **RR Rotation (Single Left Rotation):** Applied when insertion occurs in the right subtree of the right child ($BF = -2$, child $BF = -1$).
3. **LR Rotation (Double Rotation):** Insertion in the right subtree of the left child ($BF = +2$, child $BF = -1$). First Left-rotate left child, then Right-rotate parent.
4. **RL Rotation (Double Rotation):** Insertion in the left subtree of the right child ($BF = -2$, child $BF = +1$). First Right-rotate right child, then Left-rotate parent.

Step-by-Step AVL Construction Walkthrough

Construct an AVL tree by inserting keys: $10, 20, 30, 40, 50, 25$.
- Insert $10, 20$: Tree balanced.
- Insert 30: Path $10 \to 20 \to 30$ causes $BF(10) = -2$ (RR violation). Perform **RR (Left Rotation)** on 10: 20 becomes root, with children 10 (left) and 30 (right).

© Biraj

- Insert 40: Attached as right child of 30. Balanced ($BF(20) = -1$).
- Insert 50: Path $30 \to 40 \to 50$ causes $BF(30) = -2$ (RR violation). Perform **RR (Left Rotation)** on 30: 40 takes 30's position, children $30, 50$.
- Insert 25: Attached as left child of 30. Root 20 now has $BF(20) = -2$, child 40 has $BF(40) = +1$ (RL violation). Perform **RL Double Rotation**: Right-rotate 40 then Left-rotate 20. New root is 30, left subtree rooted at 20 (children $10, 25$), right subtree rooted at 40 (child 50). Tree is perfectly balanced!

Height Bound and Complexity of AVL Trees

The minimum number of nodes $N(h)$ in an AVL tree of height h satisfies the Fibonacci recurrence:

$$N(h) = N(h-1) + N(h-2) + 1, \quad N(0) = 1,\ N(1) = 2 \tag{26}$$

Solving this relation yields an asymptotic upper bound on height:

$$h < 1.44 \log_2(n+2) \implies \mathbf{h = \mathcal{O}(\log_2 n)} \tag{27}$$

Consequently, Search, Insert, and Delete operations in an AVL tree are **guaranteed strictly** $\mathcal{O}(\log \mathbf{n})$ in both average and worst cases!

8. Applications of Binary Trees: Expression Trees and Huffman Coding

8.1 Expression Trees

An **Expression Tree** is a binary tree representing an arithmetic or logical formula:

- **Leaves:** Distinct operands (variables or numerical constants).
- **Internal Nodes:** Binary operators $(+, -, *, /, \wedge)$.
- **Traversals:** Inorder traversal with parentheses yields the equivalent **Infix** formula; Preorder traversal yields **Prefix**; Postorder traversal yields **Postfix**.
- **Construction Algorithm from Postfix:** Scan postfix expression token by token using a stack of tree node pointers. For an operand, create a leaf node and push pointer. For operator $\odot$, create an operator node, pop two nodes as right and left children, and push the newly formed subtree.

8.2 Huffman Coding (Greedy Data Compression)

Huffman Algorithm for Optimal Prefix Codes

Given an alphabet of n symbols with given occurrence frequencies:

1. Create a Min-Priority Queue containing n leaf nodes, each initialized with a symbol and its frequency.
2. While the priority queue contains more than one node:
   - $left$ = Extract-Min()
   - $right$ = Extract-Min()
   - Allocate a new internal node Z with freq(Z) = freq$(left)$ + freq$(right)$.

© Biraj

- Set $Z \rightarrow \text{left} = left$ and $Z \rightarrow \text{right} = right$.
- Insert(Z) back into the Min-Priority Queue.

3. The remaining single node is the root of the **Huffman Tree**.
4. Assign binary bit `'0'` to each left branch and `'1'` to each right branch. Paths from root to leaves produce unambiguous **Prefix Codes** (no code is a prefix of another).

Time Complexity: $\mathcal{O}(\mathbf{n} \log_2 \mathbf{n})$ using a binary min-heap.

9. Multi-Way Search Trees: B-Tree and B+ Tree

9.1 B-Tree of Order m

Designed for secondary storage (disk systems) to minimize slow disk I/O operations by maximizing node branching factor.

Axiomatic Properties of B-Tree of Order m

1. Every node has at most m children and $m-1$ keys.
2. The root node has at least 2 children (unless it is a leaf).
3. Every internal node (except root) has at least $\lceil m/2 \rceil$ children and $\lceil m/2 \rceil - 1$ keys.
4. All leaf nodes reside at the exact same depth/level.
5. A non-leaf node with k children contains $k-1$ keys acting as separation boundaries.

B-Tree Algorithmic Operations: Insertion and Deletion

1. B-Tree Insertion Algorithm:

- Search to find the appropriate leaf node for key K.
- If leaf contains $< m-1$ keys, insert K into sorted position.
- **Node Splitting (Overflow):** If leaf already contains $m-1$ keys, inserting K causes overflow (m keys). Split node around the **median key** at position $\lceil m/2 \rceil$:
  - Keys left of median form a new left leaf.
  - Keys right of median form a new right leaf.
  - Median key is **promoted** to the parent node.
  - If parent overflows, propagate splitting upward recursively (if root splits, tree height increases by 1).

2. B-Tree Deletion Algorithm:

- If key is in an internal node, replace it with its inorder predecessor or successor, reducing problem to deletion from a leaf.
- If leaf has $\geq \lceil m/2 \rceil$ keys, delete directly.
- **Underflow Handling ($< \lceil m/2 \rceil - 1$ keys):**
  - **Borrowing (Rotation):** If immediate sibling has $> \lceil m/2 \rceil - 1$ keys, borrow a key via rotation through parent.
  - **Merging:** If both siblings have minimum keys, merge the node with a sibling and the intervening parent separator key (may cause underflow in parent).

© Biraj

9.2 B+ Tree and Comparison with B-Tree

- **B+ Tree Structure:** In a B+ tree, all data records and actual keys are stored **strictly in leaf nodes**. Internal nodes store only search routing keys.
- **Linked Leaves:** All leaf nodes are linked sequentially via pointers to form a doubly linked list, enabling extremely fast $\mathcal{O}(k)$ range queries.
- **Database Indexing Standard:** B+ Trees are universally used in relational database engines (MySQL, PostgreSQL, Oracle) and file systems (NTFS, ext4).

© Biraj

Quick Revision — Module 3

Linked Lists and Tree Data Structures | ES-CS301

Key Formulas, Tree Bounds and Complexity at a Glance

Binary Tree Max Nodes Level i / Total	$N_{\max}(i) = 2^i, \quad N_{\text{total}}(h) = 2^{h+1} - 1$
Leaf Nodes vs Degree-2 Nodes	$n_0 = n_2 + 1$ (Universal Binary Tree Invariant)
Binary Tree Minimum Height	$h_{\min} = \lceil \log_2(n+1) \rceil - 1 \implies \Omega(\log n)$
AVL Balance Factor	$BF(v) = h_L(v) - h_R(v) \in \{-1, 0, +1\}$
AVL Maximum Height Bound	$h < 1.44 \log_2 n \implies$ Guaranteed $\mathcal{O}(\log n)$
SLL Head Operations vs Search	Insert/Delete Head: $\mathcal{O}(1)$, Search/Tail: $\mathcal{O}(n)$
BST Search / Insert / Delete	Average: $\mathcal{O}(\log n)$, Worst (skewed): $\mathcal{O}(n)$
B-Tree Key Bounds per Node	$\lceil m/2 \rceil - 1 \leq$ keys $\leq m - 1$, Search: $\mathcal{O}(\log_m n)$

One-Line Definitions

- **Singly Linked List:** A dynamic linear data structure composed of nodes storing data and a single pointer to the successor.
- **Doubly Linked List:** A linked list where each node contains pointers to both its forward and backward adjacent nodes.
- **Circular Linked List:** A linked list whose terminal node points back to the first node instead of storing `NULL`.
- **Header Node:** A dummy lead node placed before actual data elements to simplify edge cases and eliminate head pointer updates.
- **Complete Binary Tree:** A binary tree completely filled at all levels except possibly the last, which is filled left-to-right.
- **Threaded Binary Tree:** A binary tree that replaces null leaf pointers with threads to inorder predecessors and successors.
- **Binary Search Tree (BST):** A binary tree where all left descendants are smaller and all right descendants are greater than the node.
- **AVL Tree:** A height-balanced BST where the heights of left and right subtrees of every node differ by at most 1 ($|BF| \leq 1$).

© Biraj

Frequently Asked / Viva Questions (Part 1)

Q1. State and prove the relationship between leaf nodes and degree-2 nodes in any non-empty binary tree.

In any binary tree, $n_0 = n_2 + 1$. Proof: Let total nodes be $n = n_0 + n_1 + n_2$. Total edges in a tree of n nodes is $E = n - 1$. Also, each degree-1 node contributes 1 outgoing branch and each degree-2 node contributes 2: $E = n_1 + 2n_2$. Setting $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2$ simplifies directly to $n_0 = n_2 + 1$.

Q2. Why does Inorder traversal of a Binary Search Tree produce sorted output?

Inorder traversal follows the Left-Data-Right (L-D-R) sequence. Because the BST invariant guarantees that all left subtree nodes are smaller than the root, and the root is smaller than all right subtree nodes, recursively visiting Left, then Root, then Right naturally produces a monotonically ascending sequence.

Q3. Explain the four rebalancing rotations of an AVL tree.

Single Right (LL) fixes left-heavy imbalance caused by insertion in left subtree of left child. Single Left (RR) fixes right-heavy imbalance. Left-Right (LR) double rotation fixes left-heavy node with right-heavy child via left rotation on child followed by right on parent. Right-Left (RL) performs right rotation on child followed by left on parent.

Q4. How does a B+ Tree differ fundamentally from a B-Tree?

In a B-tree, keys and associated data records are stored in both internal and leaf nodes. In a B+ tree, all data records are stored strictly in leaf nodes; internal nodes store only routing index keys. Furthermore, leaf nodes in a B+ tree are linked sequentially into a linked list for rapid range scanning.

Q5. What are the advantages of maintaining a Tail pointer in a Circular Linked List?

With only a tail pointer, the last node is directly accessed in $\mathcal{O}(1)$ via `tail`, and the first node is accessed in $\mathcal{O}(1)$ via `tail->next`. This allows insertion at the beginning and insertion at the end to both execute in strictly $\mathcal{O}(1)$ time without traversing the list.

Q6. What is the primary motivation for using Threaded Binary Trees?

In a binary tree of n nodes, $n + 1$ pointer fields are wasted as `NULL`. Threaded trees replace these null pointers with threads pointing to inorder predecessors and successors, permitting full inorder traversal without recursion or an auxiliary stack ($\mathcal{O}(1)$ extra space).

Q7. What is the worst-case time complexity of searching in a BST, and when does it occur?

The worst-case search complexity is $\mathcal{O}(n)$. This occurs when keys are inserted in strictly sorted or reverse-sorted order, causing the BST to degenerate into a skewed linear chain where height $h = n - 1$.

© Biraj

Frequently Asked / Viva Questions (Part 2)

Q8. Why are B-Trees preferred over Binary Search Trees for database indexes on disk?

Disk access is significantly slower than RAM access. A BST has high height ($\mathcal{O}(\log_2 n)$), requiring many sequential disk seeks. A B-Tree has a high branching factor m (order $m \approx 100$ to 1000), keeping height very low ($\mathcal{O}(\log_m n)$), so only 2 to 3 disk I/O reads are needed to retrieve any record.

Q9. Explain the three cases of node deletion in a Binary Search Tree.

Case 1: Leaf node (simply remove and nullify parent's pointer). Case 2: Node has one child (bypass node by connecting parent directly to child). Case 3: Node has two children (replace value with inorder successor or predecessor, then delete that node which has at most one child).

Q10. What is the purpose of a Header Node in a linked list?

A header node simplifies boundary conditions: it ensures the list is never empty (head pointer never changes) and eliminates special-case logic for inserting or deleting at the front of the list.

Summary Matrix: Comprehensive Comparison of Tree Data Structures

Tree Structure	Structural Invariant	Search Complexity	Primary Application Domain
Binary Tree	**At most 2 children per node**	$\mathcal{O}(n)$	Expression trees, hierarchical parsing, Huffman coding.
Threaded Binary Tree	**Null pointers store inorder links**	**Traversal:** $\mathcal{O}(1)$ **space**	Stackless tree traversal in memory-constrained systems.
Binary Search Tree (BST)	**Left** $<$ **Root** $<$ **Right**	**Avg:** $\mathcal{O}(\log n)$, **Worst:** $\mathcal{O}(n)$	Dynamic searching, symbol tables, dictionary ADT.
AVL Tree	**Strict balance:** $\lvert BF\rvert \leq 1$	**Guaranteed** $\mathcal{O}(\log n)$	Search-intensive in-memory datasets and lookup tables.
B-Tree	**Multi-way:** $\lceil m/2 \rceil \leq$ **children** $\leq m$	$\mathcal{O}(\log_m n)$	File systems, relational databases, storage indexing.
B+ Tree	**Data in leaves; leaves linked**	$\mathcal{O}(\log_m n)$	Database query engines, fast sequential range retrieval.

© Biraj

Module 4

Classification and Criteria of Sorting Algorithms

1. Classification and Criteria of Sorting Algorithms

Definition: Sorting Problem and Taxonomic Criteria

Given a sequence of n records $R_1, R_2, \dots, R_n$ with keys $K_1, K_2, \dots, K_n$, **sorting** is the algorithmic permutation of these records into an ordered sequence $R_{\pi(1)}, R_{\pi(2)}, \dots, R_{\pi(n)}$ such that:

$$K_{\pi(1)} \leq K_{\pi(2)} \leq \dots \leq K_{\pi(n)} \tag{28}$$

Core Taxonomic Criteria:

1. **Internal vs External Sorting:** Internal sorting holds the entire dataset concurrently in main RAM. External sorting processes massive datasets residing on secondary storage (hard drives, tapes) using phased multi-way merge passes.
2. **Space Efficiency (In-Place vs Out-of-Place):** An in-place algorithm requires strictly $\mathcal{O}(1)$ auxiliary space beyond the input buffer (e.g., Insertion Sort, Heap Sort). Out-of-place algorithms require $\mathcal{O}(n)$ auxiliary storage (e.g., Merge Sort).
3. **Stability:** A sorting algorithm is **stable** if records with identical keys preserve their relative initial input ordering:

$$K_i = K_j \text{ with } i < j \implies \pi(i) < \pi(j) \tag{29}$$

4. **Adaptivity:** An algorithm is adaptive if its runtime decreases when the input array is already partially or completely sorted (e.g., Bubble Sort with swap flag, Insertion Sort).

Theoretical Lower Bound for Comparison-Based Sorting

Any deterministic comparison-based sorting algorithm can be modeled as a binary decision tree where each internal node represents a comparison $A[i] \leq A[j]$ and each leaf represents one of the $n!$ possible permutations of the input.

For a binary tree of height h with at least $n!$ leaves:

$$2^h \geq n! \implies h \geq \log_2(n!) \tag{30}$$

Applying Stirling's approximation ($\ln(n!) \approx n \ln n - n$):

$$h \geq \sum_{k=1}^{n} \log_2 k \geq \sum_{k=n/2}^{n} \log_2 \left(\frac{n}{2}\right) = \frac{n}{2} \log_2 \left(\frac{n}{2}\right) = \boxed{\mathbf{n \log_2 n}} \tag{31}$$

© Biraj

Thus, no comparison-based sorting algorithm can achieve a worst-case time complexity faster than $\square(\mathbf{n}\log\mathbf{n})$.

2. Elementary Sorting Algorithms

2.1 Selection Sort

- **Working Principle:** In each pass $i \in [0, n-2]$, find the minimum element in unsorted subarray $A[i \dots n-1]$ and swap it with $A[i]$.
- **Comparisons:** Total comparisons are fixed for all input distributions:

$$C(n) = \sum_{i=0}^{n-2}(n-1-i) = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2} = \square(\mathbf{n^2}) \tag{32}$$

- **Swaps:** At most $n-1$ swaps ($\mathcal{O}(n)$ writes) — ideal when write operations are physically expensive (e.g., EEPROM/Flash memory).
- **Properties:** In-place ($\mathcal{O}(1)$ space), **Unstable** (e.g., sorting $[4_a, 4_b, 2]$ swaps 4_a with 2, placing 4_a after 4_b).

2.2 Bubble Sort

- **Working Principle:** Iteratively scan the array, comparing adjacent pairs $(A[j], A[j+1])$ and swapping them if out of order, bubbling the largest item to the end in each pass.
- **Optimization:** Maintain a boolean flag `swapped`. If no swaps occur in a full pass, the array is already sorted and the algorithm terminates immediately.
- **Complexity:** Best-case: $\mathcal{O}(\mathbf{n})$ (with flag on sorted input). Average and Worst-case: $\mathcal{O}(\mathbf{n^2})$ comparisons and swaps.
- **Properties:** In-place ($\mathcal{O}(1)$ space), **Stable**, Adaptive.

2.3 Insertion Sort

- **Working Principle:** Simulates how cards are sorted in hand: incrementally pick $A[i]$ and insert it into its correct relative position within the sorted subarray $A[0 \dots i-1]$ by shifting larger elements one position to the right.
- **Complexity Analysis:**
  - **Best-Case (Sorted Input):** Only 1 comparison per element, 0 shifts $\implies \mathcal{O}(\mathbf{n})$ time.
  - **Worst-Case (Reverse-Sorted Input):** Max shifts: $\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2} \implies \mathcal{O}(\mathbf{n^2})$ time.
  - **Average-Case:** $\frac{n(n-1)}{4}$ shifts $\implies \mathcal{O}(\mathbf{n^2})$ time.
- **Properties:** In-place ($\mathcal{O}(1)$ space), **Stable**, Online (can sort incoming streaming data), highly optimal for small arrays ($n \leq 16$).

© Biraj

3. Advanced Divide-and-Conquer & Tree-Based Sorting

3.1 Quick Sort (Partition-Exchange Sort)

Quick Sort Principle and Partitioning Strategy

Quick Sort operates on the **Divide-and-Conquer** paradigm:

1. **Partition:** Select a pivot element P. Rearrange array such that all elements $\leq P$ precede P, and all elements $\geq P$ succeed P. The pivot P settles into its final sorted position q.
2. **Recurse:** Recursively apply Quick Sort to subarrays $A[\text{low} \dots q-1]$ and $A[q+1 \dots \text{high}]$.

Lomuto Partition Algorithm $(A, \textbf{low}, \textbf{high})$**:**

```
pivot = A[high]; i = low - 1;
for j = low to high - 1 do:
    if (A[j] <= pivot) then:
        i = i + 1; swap(A[i], A[j]);
swap(A[i + 1], A[high]);
return (i + 1);
```

Quick Sort Recurrence Relations and Asymptotic Analysis

1. Best-Case Analysis (Balanced Partition): Pivot consistently bisects the array into two equal halves of size $n/2$:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \implies \mathbf{T(n) = \square(n \log_2 n)} \tag{33}$$

2. Worst-Case Analysis (Extremely Unbalanced Partition): Occurs when the array is already sorted or reverse-sorted and the first or last element is chosen as pivot, yielding subproblems of size 0 and $n-1$:

$$T(n) = T(n-1) + T(0) + \Theta(n) = T(n-1) + cn = c\sum_{i=1}^{n} i \implies \mathbf{T(n) = \square(n^2)} \tag{34}$$

3. Average-Case Analysis: Assuming all permutations equally probable: $\mathcal{O}(\mathbf{n \log n})$.

Auxiliary Space: $\mathcal{O}(\log n)$ stack frames for balanced recursion; degrades to $\mathcal{O}(n)$ in worst case. Quick Sort is **Unstable**.

3.2 Merge Sort

Merge Sort Algorithm and Guaranteed Bounds

Merge Sort recursively divides the array into two equal halves until single-element subarrays are reached, then merges them in sorted order using an auxiliary buffer:

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) \quad \forall n > 1, \quad T(1) = 0 \tag{35}$$

© Biraj

By the Master Theorem (Case 2: $a = 2, b = 2, f(n) = \Theta(n) = \Theta(n^{\log_2 2})$):

$$\mathbf{T(n) = \square(n \log_2 n)} \quad \text{in Best, Average, and Worst Cases!} \tag{36}$$

Two-Way Merge Algorithm (A, low, mid, high):

```
i = low; j = mid + 1; k = low;
while (i <= mid && j <= high) {
    if (A[i] <= A[j]) { B[k] = A[i]; i++; }
    else { B[k] = A[j]; j++; }
    k++;
}
while (i <= mid) { B[k] = A[i]; i++; k++; }
while (j <= high) { B[k] = A[j]; j++; k++; }
for (p = low; p <= high; p++) A[p] = B[p];
```

Key Trade-offs:
- **Guaranteed Performance:** Never degrades to $\mathcal{O}(n^2)$.
- **Space Overhead:** Requires $\mathcal{O}(\mathbf{n})$ auxiliary memory array.
- **Stability: Stable** (ideal for complex multi-attribute sorting).
- **Linked Lists:** Does not require random access; sorts linked lists in $\mathcal{O}(n \log n)$ time with $\mathcal{O}(1)$ extra space.

3.3 Heap Sort

A **Binary Heap** is a Complete Binary Tree mapped sequentially into an array $A[0 \ldots n-1]$ satisfying the Max-Heap property: $A[\text{parent}] \geq A[\text{child}]$.

- Array Index Mapping: $\text{Parent}(i) = \lfloor (i-1)/2 \rfloor$, $\text{Left}(i) = 2i + 1$, $\text{Right}(i) = 2i + 2$.

Heap Sort Procedures and Linear Build-Heap Proof

1. Max-Heapify(A, i, n) ($\mathcal{O}(\log n)$):

```
largest = i; left = 2*i + 1; right = 2*i + 2;
if (left < n && A[left] > A[largest]) largest = left;
if (right < n && A[right] > A[largest]) largest = right;
if (largest != i) {
    swap(A[i], A[largest]);
    Max-Heapify(A, largest, n);
}
```

2. Build-Max-Heap ($O(n)$): Calls `Max-Heapify` on all internal nodes from $\lfloor n/2 \rfloor - 1$ down to 0:

$$\sum_{h=0}^{\lfloor \log_2 n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \mathcal{O}(h) = \mathcal{O}\left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) = \mathcal{O}(n \cdot 2) = \mathcal{O}(\mathbf{n}) \tag{37}$$

3. Heap Sort Algorithm ($\mathcal{O}(n \log n)$):
1. Build a Max-Heap from input array: $\mathcal{O}(n)$.
2. For $i = n - 1$ down to 1:
   - Swap root $A[0]$ (current maximum) with $A[i]$.

© Biraj

- Reduce heap size by 1.
- Call `Max-Heapify`$(A, 0, i)$: $\mathcal{O}(\log n)$.

Complexity: Best, Average, Worst time strictly $\mathcal{O}(\mathbf{n} \log \mathbf{n})$. In-place ($\mathcal{O}(\mathbf{1})$ space). **Unstable**.

3.4 Comprehensive Sorting Comparison Matrix

Algorithm	Best	Average	Worst	Space	Stable?	Primary Use Case
Selection Sort	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	**No**	Minimal write operations.
Bubble Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	**Yes**	Educational; nearly sorted data.
Insertion Sort	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$	$\mathcal{O}(1)$	**Yes**	Small lists ($n \leq 16$); streaming.
Quick Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n^2)$	$\mathcal{O}(\log n)$	**No**	Fastest general in-memory sort.
Merge Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n)$	**Yes**	Linked lists; external sorting.
Heap Sort	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(n \log n)$	$\mathcal{O}(1)$	**No**	Guaranteed $\mathcal{O}(n \log n)$ in-place.

4. Hashing Techniques and Collision Resolution

Definition: Hashing, Hash Table and Load Factor

Hashing is a transformation technique that maps large variable keys K from a universe U into fixed integer slot indices in a **Hash Table** $T[0 \ldots m-1]$ via a mathematical **Hash Function** $h(K)$:

$$h : U \longrightarrow \{0, 1, \ldots, m-1\} \tag{38}$$

The **Load Factor** (α) is the ratio of stored keys n to table capacity m:

$$\alpha = \frac{n}{m} \tag{39}$$

A **Collision** occurs when two distinct keys hash to the same slot: $h(K_1) = h(K_2)$ for $K_1 \neq K_2$.

4.1 Standard Hash Functions

1. **Division Method:** $h(K) = K \bmod m$. The table size m should be a prime number not close to a power of 2 or 10 to minimize clustering.
2. **Mid-Square Method:** Square the key K^2, and extract r middle digits/bits as the index. Table size $m = 2^r$.

© Biraj

3. **Folding Method:** Partition K into parts matching index digit length, then sum parts:
   - **Fold-Shift:** Add components directly: $h(K) = (p_1 + p_2 + \dots) \bmod m$.
   - **Fold-Boundary:** Reverse the digits of the outer parts before adding.
4. **Multiplication Method:** $h(K) = \lfloor m(KA \bmod 1) \rfloor$, where $0 < A < 1$ (Knuth recommends $A \approx \frac{\sqrt{5}-1}{2} \approx 0.618033$).

4.2 Collision Resolution Techniques

1. **Separate Chaining (Open Hashing):** Each table slot is a header pointer to a linked list of records hashing to that slot.
   - Can accommodate $\alpha > 1$ (table never gets "full").
   - Average search time: $\mathcal{O}(1 + \alpha)$.
2. **Open Addressing (Closed Hashing):** All records reside directly in the table array ($m \geq n \implies \alpha \leq 1$). If collision occurs, search alternate slots via a systematic probe sequence $h(K, i)$:
   - **Linear Probing:** $h(K, i) = (h'(K) + i) \bmod m$. Suffers from **Primary Clustering** (adjacent occupied slots merge into long chains, degrading search to $\mathcal{O}(n)$).
   - **Quadratic Probing:** $h(K, i) = (h'(K) + c_1 i + c_2 i^2) \bmod m$. Eliminates primary clustering but suffers from **Secondary Clustering** (keys hashing to same initial slot trace identical probe paths).
   - **Double Hashing:** $h(K, i) = (h_1(K) + i \cdot h_2(K)) \bmod m$. Probe offset depends on the key itself ($h_2(K)$). Completely eliminates both primary and secondary clustering! Condition: $\gcd(h_2(K), m) = 1$.

Exam Numerical: Collision Resolution Walkthrough

Problem: Insert keys $K = \{23, 43, 13, 27\}$ into a hash table of size $m = 7$ using $h'(K) = K \bmod 7$.

1. Under Linear Probing ($h(K, i) = (h'(K) + i) \bmod 7$)**:**

- $K = 23$: $h'(23) = 23 \bmod 7 = 2$. Slot 2 is empty $\implies T[2] = 23$.
- $K = 43$: $h'(43) = 43 \bmod 7 = 1$. Slot 1 is empty $\implies T[1] = 43$.
- $K = 13$: $h'(13) = 13 \bmod 7 = 6$. Slot 6 is empty $\implies T[6] = 13$.
- $K = 27$: $h'(27) = 27 \bmod 7 = 6$. **Collision at 6!** Probe $i = 1$: $(6 + 1) \bmod 7 = 0$. Slot 0 is empty $\implies T[0] = 27$.

Final Table: $T[0] = 27,\ T[1] = 43,\ T[2] = 23,\ T[6] = 13$.

5. Graph Theory, Representations and Traversals

Definition: Graph Data Structure and Core Terminologies

A **Graph** $G = (V, E)$ consists of a non-empty finite set of vertices V and a set of edges E connecting pairs of vertices.

© Biraj

- **Handshaking Lemma:** In any undirected graph:

$$\sum_{v \in V} \deg(v) = 2|E| \tag{40}$$

  Corollary: In any graph, the number of vertices with odd degree is always even.
- **Complete Graph K_n:** Every pair of distinct vertices is connected by a unique edge: $|E| = \frac{n(n-1)}{2}$.
- **Connected Component:** A maximal connected subgraph of an undirected graph.
- **Strongly Connected Digraph:** A directed graph where there exists a directed path between every ordered pair of vertices (u, v).

5.1 Graph Storage Representations

Representation	Space Complexity	Edge Lookup (u, v)	Optimal Suitability
Adjacency Matrix	$\mathcal{O}(V^2)$	$\mathcal{O}(1)$	Dense graphs ($\|E\| \approx \|V\|^2$); fast edge tests.
Adjacency List	$\mathcal{O}(V + E)$	$\mathcal{O}(\deg(u))$	Sparse graphs ($\|E\| \ll \|V\|^2$); DFS/BFS algorithms.
Incidence Matrix	$\mathcal{O}(V \cdot E)$	$\mathcal{O}(E)$	Theoretical network flows; hypergraphs.

5.2 Graph Search and Traversal Algorithms

Breadth-First Search (BFS) and Depth-First Search (DFS) Algorithms

1. Algorithm BFS(G, s):

```
initialize boolean visited[V] = false, Queue Q;
visited[s] = true; Enqueue(Q, s);
while (!IsEmpty(Q)) {
    u = Dequeue(Q); print(u);
    for each vertex v adjacent to u:
        if (!visited[v]) {
            visited[v] = true; Enqueue(Q, v);
        }
}
```

Time: $\mathcal{O}(\mathbf{V} + \mathbf{E})$ (Adjacency List), $\mathcal{O}(V^2)$ (Matrix) Space: $\mathcal{O}(V)$

2. Algorithm DFS(G, u):

```
visited[u] = true; print(u);
for each vertex v adjacent to u:
    if (!visited[v]) {
        DFS(G, v); // Recursive tree exploration
    }
```


© Biraj

Time: $\mathcal{O}(\mathbf{V}+\mathbf{E})$ Space: $\mathcal{O}(V)$ (System recursion stack)
Edge Classification in DFS: Tree edges (forming DFS forest), Back edges (to ancestor $\implies$ cycle detected!), Forward edges (to descendant), Cross edges (between distinct subtrees).

© Biraj

Quick Revision — Module 4

Sorting, Hashing and Graph Theory | ES-CS301

Key Formulas, Recurrences and Complexities at a Glance

Comparison Sort Lower Bound	$\Omega(n \log_2 n)$ via Decision Tree Height $h \geq \log_2(n!)$
Merge Sort Recurrence	$T(n) = 2T(n/2) + \Theta(n) \implies \Theta(n \log_2 n)$ (Guaranteed)
Quick Sort Recurrences	$T_{\text{avg}}(n) = 2T(n/2) + \Theta(n) \implies \Theta(n \log_2 n)$, $T_{\text{worst}}(n) = \Theta(n^2)$
Linear Build-Heap Bound	$\sum_{h=0}^{\log n} \frac{n}{2^{h+1}} \mathcal{O}(h) = \mathcal{O}(n)$ (Linear Time Proof)
Hash Table Load Factor	$\alpha = n/m$ ($\alpha \leq 1$ for open addressing, $\alpha > 1$ for chaining)
Double Hashing Probe Offset	$h(K, i) = (h_1(K) + i \cdot h_2(K)) \bmod m$ ($h_2(K)$ coprime to m)
Graph Handshaking Lemma	$\sum_{v \in V} \deg(v) = 2\lvert E \rvert \implies$ Number of odd-degree vertices is even
BFS / DFS Traversal Time	$\mathcal{O}(V+E)$ (Adjacency List), $\mathcal{O}(V^2)$ (Adjacency Matrix)

One-Line Definitions

- **Stable Sort:** A sorting algorithm that strictly preserves the relative input order of records with duplicate keys.
- **In-Place Sort:** A sorting algorithm requiring strictly $\mathcal{O}(1)$ auxiliary memory space beyond the input buffer.
- **Max-Heap:** A complete binary tree where key of every parent node is greater than or equal to its children ($A[\text{parent}(i)] \geq A[i]$).
- **Primary Clustering:** The undesirable tendency in linear probing where occupied adjacent slots coalesce into large continuous blocks.
- **Double Hashing:** An open addressing collision resolution technique where probe step size is calculated by an independent secondary hash function.
- **Adjacency List:** An array of linked lists where list i stores all adjacent neighbors of vertex i, requiring optimal $\mathcal{O}(V + E)$ space.
- **Breadth-First Search (BFS):** Concentric layer-by-layer graph traversal implemented via a FIFO queue; yields shortest paths in unweighted graphs.
- **Depth-First Search (DFS):** Deep branch backtracking traversal using recursion/stack; detects cycles definitively via back edges.

© Biraj

Frequently Asked / Viva Questions (Part 1)

Q1. Why can comparison-based sorting algorithms not execute faster than $\mathcal{O}(n \log n)$ in the worst case?

Every comparison sort can be modeled as a decision tree with $n!$ leaves corresponding to all permutations. A binary tree of height h has at most 2^h leaves. Therefore, $2^h \geq n! \implies h \geq \log_2(n!) \approx n \log_2 n - n \log_2 e = \Omega(n \log n)$. The height represents the worst-case number of comparisons.

Q2. Under what conditions does Quick Sort degrade to its worst-case $\mathcal{O}(n^2)$ time?

When the partition is maximally unbalanced (e.g., one subproblem of size $n-1$ and one of size 0). This occurs when the array is already sorted or reverse-sorted and the first or last element is consistently selected as the pivot.

Q3. Why is Heap construction (`Build-Max-Heap`) $\mathcal{O}(n)$ rather than $\mathcal{O}(n \log n)$?

Although `Max-Heapify` takes $\mathcal{O}(h)$ time, most nodes in a binary tree reside near the leaves with very small heights h. Summing the cost across all heights $\sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^{h+1}} h = \frac{n}{2} \sum \frac{h}{2^h} = \frac{n}{2} \cdot 2 = \mathcal{O}(n)$.

Q4. Compare Separate Chaining and Open Addressing for collision resolution.

Separate Chaining uses external linked lists for colliding elements; it never suffers from table overflow and handles load factor $\alpha > 1$. Open Addressing stores all elements directly within the array table ($\alpha \leq 1$); it eliminates pointer memory overhead but requires collision probing and can suffer from clustering.

Q5. What is the difference between Primary Clustering and Secondary Clustering?

Primary clustering occurs in linear probing when different keys hashing to adjacent slots merge into a single long contiguous chain. Secondary clustering occurs in quadratic probing when two distinct keys having the same initial hash index trace the exact same probe sequence.

Q6. State the Handshaking Lemma and explain its primary corollary.

The Handshaking Lemma states that in any undirected graph, $\sum_{v \in V} \deg(v) = 2|E|$, because each edge contributes 1 degree to two incident vertices. The corollary is that the number of vertices with odd degree in any graph must be strictly even.

Q7. Compare Adjacency Matrix and Adjacency List representations of a graph.

Adjacency Matrix uses $\mathcal{O}(V^2)$ space and checks edge existence in $\mathcal{O}(1)$ time; it is optimal for dense graphs ($|E| \approx |V|^2$). Adjacency List uses $\mathcal{O}(V+E)$ space and finds neighbors in $\mathcal{O}(\deg(v))$ time; it is optimal for sparse graphs ($|E| \ll |V|^2$).

© Biraj

Frequently Asked / Viva Questions (Part 2)

Q8. How does DFS detect cycles in a directed graph?

During DFS traversal, vertices are colored White (unvisited), Gray (in active recursion stack), and Black (completely processed). If an edge (u, v) encounters a Gray vertex v, that edge is a **Back Edge**, which definitively indicates the presence of a directed cycle.

Q9. Why is Merge Sort preferred over Quick Sort for sorting linked lists?

In linked lists, random access is not available, which makes Quick Sort partitioning slow. Merge Sort divides a linked list in $\mathcal{O}(n)$ using the slow-fast pointer technique, and merges two sorted lists in $\mathcal{O}(n)$ time and strictly $\mathcal{O}(1)$ auxiliary space without requiring contiguous arrays.

Q10. What is Topological Sorting, and what type of graph is required?

Topological sorting is a linear ordering of vertices such that for every directed edge (u, v), vertex u appears before v. It is strictly defined only on a **Directed Acyclic Graph (DAG)**.

Summary Matrix: Comprehensive Comparison of Sorting, Hashing and Graphs

Technique / Topic	Core Mechanism	Time / Space Complexity	Key Practical Application
Quick Sort	**Divide-and-Conquer; Pivot partitioning**	**Avg:** $\mathcal{O}(n \log n)$, **Worst:** $\mathcal{O}(n^2)$ **Space:** $\mathcal{O}(\log n)$ **stack**	Fastest general in-memory primitive data sorting.
Merge Sort	**Divide-and-Conquer; Two-finger merge**	**Best/Avg/Worst:** $\mathcal{O}(n \log n)$ **Space:** $\mathcal{O}(n)$ **auxiliary**	External storage sorting, linked list sorting.
Heap Sort	**Binary Max-Heap; Root-leaf exchange**	**Guaranteed** $\mathcal{O}(n \log n)$ **all cases Space:** $\mathcal{O}(1)$ **in-place**	Systems where guaranteed $\mathcal{O}(n \log n)$ and minimal memory are required.
Double Hashing	**Secondary hash step;** $h_1(K) + i \cdot h_2(K)$	**Search:** $\mathcal{O}(1)$ **avg, Worst:** $\mathcal{O}(m)$ **Space:** $\mathcal{O}(m)$ **array**	Hash tables requiring zero primary/secondary clustering.
BFS	**FIFO Queue; Level-order expansion**	$\mathcal{O}(V + E)$ **(with Adj List) Space:** $\mathcal{O}(V)$ **queue**	Shortest paths in unweighted graphs, web crawlers.
DFS	**LIFO Stack / Recursion; Deep path exploration**	$\mathcal{O}(V + E)$ **(with Adj List) Space:** $\mathcal{O}(V)$ **stack**	Cycle detection, topological sorting, connected components.

© Biraj